

**Posts and Telecommunications Institute of Technology**



**ASSIGNMENT 2**

**INTELLIGENT SYSTEM DEVELOPMENT**

|                   |                    |
|-------------------|--------------------|
| <b>Lecturer</b>   | : TRẦN ĐÌNH QUẾ    |
| <b>Student</b>    | : MAI ANH TUẤN ANH |
| <b>Student ID</b> | : B23DCCE005       |
| <b>Class</b>      | : E23CNPM02        |

*Hà Nội, August 2026*

# A. Application 1 — Diabetes Prediction

## 1 Problem Description

The objective of this application is to predict whether a patient is diagnosed with diabetes using demographic, lifestyle, medical-history, and clinical health indicators. The target variable is `diagnosed_diabetes`, where 0 represents No Diabetes and 1 represents Diabetes. The task is therefore formulated as a binary classification problem.

Let  $X$  denote the patient features and  $y$  denote the target:

$$\begin{aligned} X &= \text{patient features} \\ y &= \text{diagnosed\_diabetes} \end{aligned}$$

For one patient, the input is represented as:

$$x_i = [x_{i1}, x_{i2}, \dots, x_{id}]^T \in \mathbb{R}^d$$

and the complete dataset can be represented as:

$$X \in \mathbb{R}^{N \times d}.$$

The objective is to learn a function

$$f: X \rightarrow y$$

that can assign the appropriate diabetes class to an unseen patient.

## 2 Kaggle Dataset

The selected dataset is the **Diabetes Health Indicators Dataset**, obtained from Kaggle:

**Kaggle URL:**

<https://www.kaggle.com/datasets/mohankrishnathalla/diabetes-health-indicators-dataset>

The dataset contains approximately **100,000 patient records** and includes demographic, lifestyle, medical-history, and clinical health indicators. Representative variables include age, gender, ethnicity, education, income, employment status, smoking status, physical activity, BMI, blood pressure, cholesterol, glucose, insulin, and HbA1c.

The original dataset contains 31 columns. For the predictive task, `diagnosed_diabetes` is used as the target. Variables potentially derived from diabetes status, particularly `diabetes_stage` and `diabetes_risk_score`, are excluded from the model input to reduce the risk of target leakage.

### Dataset summary

| Item                | Value                              |
|---------------------|------------------------------------|
| Dataset             | Diabetes Health Indicators Dataset |
| Source              | Kaggle                             |
| Observations        | 100,000                            |
| Original columns    | 31                                 |
| Predictive features | 28                                 |
| Target              | <code>diagnosed_diabetes</code>    |
| Task                | Binary classification              |

The final 28 predictive features comprise 22 numerical variables and 6 categorical variables.

### Feature groups

The selected variables represent several aspects of the patient's health profile.

#### Feature group Examples

|             |                                                                                                                   |
|-------------|-------------------------------------------------------------------------------------------------------------------|
| Demographic | age, gender, ethnicity, <code>education_level</code> , <code>income_level</code> , <code>employment_status</code> |
|-------------|-------------------------------------------------------------------------------------------------------------------|

## Feature group Examples

|                 |                                                                                                                                                    |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Lifestyle       | smoking_status, alcohol_consumption_per_week,<br>physical_activity_minutes_per_week, diet_score,<br>sleep_hours_per_day, screen_time_hours_per_day |
| Medical history | family_history_diabetes, hypertension_history,<br>cardiovascular_history                                                                           |
| Anthropometric  | bmi, waist_to_hip_ratio                                                                                                                            |
| Vital signs     | systolic_bp, diastolic_bp, heart_rate                                                                                                              |
| Lipid profile   | cholesterol_total, hdl_cholesterol, ldl_cholesterol, triglycerides                                                                                 |
| Metabolic       | glucose_fasting, glucose_postprandial, insulin_level, hba1c                                                                                        |

## 3. Data Understanding

The raw CSV was loaded into a Pandas DataFrame and inspected through its shape, schema, descriptive statistics, missing-value counts, duplicate counts, and target distribution.

```
[30]: print("Pandas version:", pd.__version__)  
      print("NumPy version:", np.__version__)  
  
      DATA_PATH = "../../DATA/diabetes_dataset.csv"
```

3

```
if not os.path.exists(DATA_PATH):  
    raise FileNotFoundError(  
        f"Cannot find '{DATA_PATH}'. "  
        "Please place the Kaggle CSV file in the notebook directory."  
    )  
  
df_raw = pd.read_csv(DATA_PATH)  
  
# Keep untouched raw data  
df = df_raw.copy()  
  
print("Dataset loaded successfully.")  
print("Shape:", df.shape)
```

```
Pandas version: 3.0.3  
NumPy version: 2.4.6  
Dataset loaded successfully.  
Shape: (100000, 31)
```

The original dataset has:

$$100,000 \times 31$$

entries. After removing the target and two variables considered unsuitable for prediction because of their relationship with diabetes status, the predictive matrix contains:

$$X \in \mathbb{R}^{100000 \times 28}.$$

The dataset contains numerical and categorical variables, requiring different preprocessing strategies before model training.

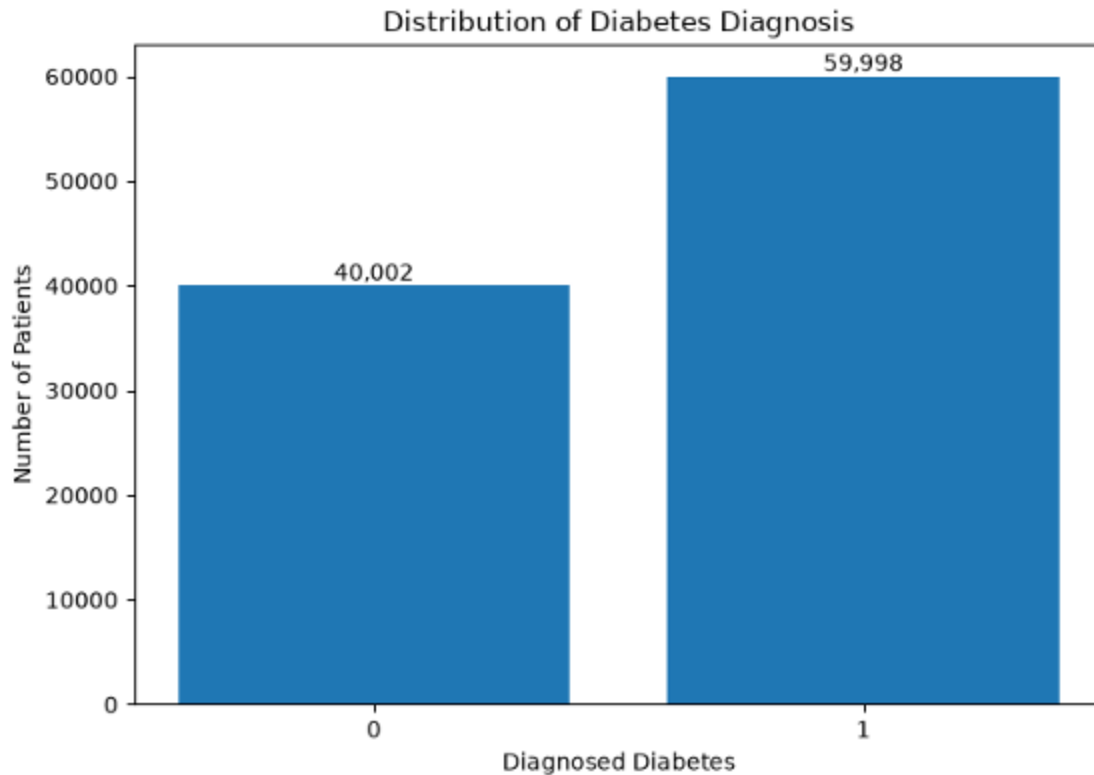
### **Target distribution**

The target contains:

- + 40,002 No Diabetes observations;
- + 59,998 Diabetes observations.

Thus, approximately 40% of the records belong to class 0 and 60% belong to class 1 => The classes are therefore not perfectly balanced, which justifies the use of `class_weight="balanced"` in several classifiers and the use of class-sensitive metrics in evaluation.

### **Distribution of the diabetes target**



**Interpretation:** The dataset contains more diabetes-positive observations than diabetes-negative observations. Consequently, accuracy alone is insufficient for judging whether the model identifies positive cases effectively.

## Data types

Data columns (total 31 columns):

| #  | Column                             | Non-Null Count  | Dtype   |
|----|------------------------------------|-----------------|---------|
| 0  | age                                | 100000 non-null | int64   |
| 1  | gender                             | 100000 non-null | str     |
| 2  | ethnicity                          | 100000 non-null | str     |
| 3  | education_level                    | 100000 non-null | str     |
| 4  | income_level                       | 100000 non-null | str     |
| 5  | employment_status                  | 100000 non-null | str     |
| 6  | smoking_status                     | 100000 non-null | str     |
| 7  | alcohol_consumption_per_week       | 100000 non-null | int64   |
| 8  | physical_activity_minutes_per_week | 100000 non-null | int64   |
| 9  | diet_score                         | 100000 non-null | float64 |
| 10 | sleep_hours_per_day                | 100000 non-null | float64 |
| 11 | screen_time_hours_per_day          | 100000 non-null | float64 |
| 12 | family_history_diabetes            | 100000 non-null | int64   |
| 13 | hypertension_history               | 100000 non-null | int64   |
| 14 | cardiovascular_history             | 100000 non-null | int64   |
| 15 | bmi                                | 100000 non-null | float64 |
| 16 | waist_to_hip_ratio                 | 100000 non-null | float64 |
| 17 | systolic_bp                        | 100000 non-null | int64   |
| 18 | diastolic_bp                       | 100000 non-null | int64   |
| 19 | heart_rate                         | 100000 non-null | int64   |
| 20 | cholesterol_total                  | 100000 non-null | int64   |
| 21 | hdl_cholesterol                    | 100000 non-null | int64   |
| 22 | ldl_cholesterol                    | 100000 non-null | int64   |
| 23 | triglycerides                      | 100000 non-null | int64   |
| 24 | glucose_fasting                    | 100000 non-null | int64   |
| 25 | glucose_postprandial               | 100000 non-null | int64   |
| 26 | insulin_level                      | 100000 non-null | float64 |
| 27 | hba1c                              | 100000 non-null | float64 |
| 28 | diabetes_risk_score                | 100000 non-null | float64 |
| 29 | diabetes_stage                     | 100000 non-null | str     |

After feature selection, the predictive variables consist of:

+ 22 numerical features

+ 6 categorical features

This distinction directly determines the preprocessing design used later in the pipeline.

## Duplicate analysis

```
[34]: duplicate_count = df.duplicated().sum()

print("Number of duplicated rows:", duplicate_count)

if duplicate_count > 0:
    duplicate_percentage = (
```

11

```
        duplicate_count / len(df) * 100
    )
    print(
        f"Percentage of duplicated rows: "
        f"{duplicate_percentage:.2f}%"
    )

    display(
        df[df.duplicated(keep=False)]
        .head(20)
    )
```

Number of duplicated rows: 0

=> The duplicate check found is exact 0 duplicated rows. Therefore, no duplicate removal was required.

## Missing values

```

: quality_summary = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "missing_count": df.isna().sum(),
    "missing_percentage": (
        df.isna().mean() * 100
    ).round(2),
    "unique_values": df.nunique(dropna=False)
})

display(
    quality_summary.sort_values(
        "missing_count",
        ascending=False
    )
)

```

|                                    | dtype   | missing_count | \ |
|------------------------------------|---------|---------------|---|
| age                                | int64   | 0             |   |
| gender                             | str     | 0             |   |
| ethnicity                          | str     | 0             |   |
| education_level                    | str     | 0             |   |
| income_level                       | str     | 0             |   |
| employment_status                  | str     | 0             |   |
| smoking_status                     | str     | 0             |   |
| alcohol_consumption_per_week       | int64   | 0             |   |
| physical_activity_minutes_per_week | int64   | 0             |   |
| diet_score                         | float64 | 0             |   |
| sleep_hours_per_day                | float64 | 0             |   |
| screen_time_hours_per_day          | float64 | 0             |   |
| family_history_diabetes            | int64   | 0             |   |
| hypertension_history               | int64   | 0             |   |
| cardiovascular_history             | int64   | 0             |   |
| bmi                                | float64 | 0             |   |
| waist_to_hip_ratio                 | float64 | 0             |   |
| systolic_bp                        | int64   | 0             |   |
| diastolic_bp                       | int64   | 0             |   |
| heart_rate                         | int64   | 0             |   |
| cholesterol_total                  | int64   | 0             |   |
| hdl_cholesterol                    | int64   | 0             |   |
| ldl_cholesterol                    | int64   | 0             |   |
| triglycerides                      | int64   | 0             |   |
| glucose_fasting                    | int64   | 0             |   |
| glucose_postprandial               | int64   | 0             |   |
| insulin_level                      | float64 | 0             |   |
| hba1c                              | float64 | 0             |   |
| diabetes_risk_score                | float64 | 0             |   |
| diabetes_stage                     | str     | 0             |   |
| diagnosed_diabetes                 | int64   | 0             |   |

The raw dataset contains no missing values in the inspected columns. Nevertheless, the final preprocessing pipeline retains imputers so that the deployed system remains robust to missing values in future unseen inputs.

=> The numerical branch uses median imputation, while the categorical branch uses most-frequent imputation.

## Invalid values

Domain checks identified potentially invalid observations in several clinical variables. The most notable cases were:

|    | feature              | allowed_min | allowed_max | invalid_count | \ |
|----|----------------------|-------------|-------------|---------------|---|
| 0  | age                  | 18          | 90          | 0             |   |
| 1  | bmi                  | 15          | 45          | 0             |   |
| 2  | systolic_bp          | 90          | 180         | 0             |   |
| 3  | diastolic_bp         | 60          | 120         | 2760          |   |
| 4  | heart_rate           | 50          | 120         | 808           |   |
| 5  | glucose_fasting      | 70          | 250         | 97            |   |
| 6  | glucose_postprandial | 90          | 350         | 1129          |   |
| 7  | insulin_level        | 2           | 50          | 0             |   |
| 8  | hba1c                | 4           | 14          | 0             |   |
| 9  | cholesterol_total    | 120         | 300         | 1999          |   |
| 10 | hdl_cholesterol      | 20          | 100         | 0             |   |
| 11 | ldl_cholesterol      | 50          | 200         | 294           |   |

13

|    |               |    |     |      |
|----|---------------|----|-----|------|
| 12 | triglycerides | 50 | 500 | 5253 |
|----|---------------|----|-----|------|

No invalid observations were detected for age, BMI, insulin level, HbA1c, or HDL cholesterol under the selected domain rules.

=> These values were treated as data-quality findings rather than automatically deleted observations. This avoids assuming that every extreme value represents an error.

## Outlier analysis

An observation  $x$  is considered a statistical outlier if it satisfies either of the following conditions:

**Lower outlier:**

$$x < Q_1 - 1.5 \times IQR$$

**Upper outlier:**

$$x > Q_3 + 1.5 \times IQR$$

$$IQR = Q_3 - Q_1$$

where:

- $Q_1$  is the **first quartile (25th percentile)**
- $Q_3$  is the **third quartile (75th percentile)**

```
numeric_columns = df.select_dtypes(  
    include=np.number  
) .columns.tolist()  
  
# The target is excluded from feature outlier analysis  
outlier_features = [  
    col for col in numeric_columns  
    if col != "diagnosed_diabetes"  
]  
  
outlier_results = []  
  
for col in outlier_features:  
  
    series = df[col].dropna()  
  
    if series.empty:  
        continue  
  
    q1 = series.quantile(0.25)  
    q3 = series.quantile(0.75)  
  
    iqr = q3 - q1  
  
    lower_bound = q1 - 1.5 * iqr  
    upper_bound = q3 + 1.5 * iqr
```

```

mask = (
    (df[col] < lower_bound) |
    (df[col] > upper_bound)
)

count = int(mask.sum())

outlier_results.append({
    "feature": col,
    "Q1": q1,
    "Q3": q3,
    "IQR": iqr,
    "lower_bound": lower_bound,
    "upper_bound": upper_bound,
    "outlier_count": count,
    "outlier_percentage": round(
        count / len(df) * 100,
        2
    )
})

outlier_df = pd.DataFrame(
    outlier_results
).sort_values(
    "outlier_percentage",
    ascending=False
)

display(outlier_df)

```

Outliers were examined using the IQR-based method. The analysis was used primarily to understand extreme observations and assess whether they could plausibly represent genuine patient variation. The notebook therefore does not perform indiscriminate outlier removal.

|    | feature                            | Q1     | Q3     | IQR    | lower_bound \ |
|----|------------------------------------|--------|--------|--------|---------------|
| 6  | family_history_diabetes            | 0.00   | 0.00   | 0.00   | 0.000         |
| 8  | cardiovascular_history             | 0.00   | 0.00   | 0.00   | 0.000         |
| 2  | physical_activity_minutes_per_week | 57.00  | 160.00 | 103.00 | -97.500       |
| 22 | diabetes_risk_score                | 23.80  | 35.60  | 11.80  | 6.100         |
| 4  | sleep_hours_per_day                | 6.30   | 7.70   | 1.40   | 4.200         |
| 13 | heart_rate                         | 64.00  | 75.00  | 11.00  | 47.500        |
| 9  | bmi                                | 23.20  | 28.00  | 4.80   | 16.000        |
| 18 | glucose_fasting                    | 102.00 | 120.00 | 18.00  | 75.000        |
| 12 | diastolic_bp                       | 70.00  | 81.00  | 11.00  | 53.500        |
| 19 | glucose_postprandial               | 139.00 | 181.00 | 42.00  | 76.000        |
| 21 | hba1c                              | 5.97   | 7.07   | 1.10   | 4.320         |
| 15 | hdl_cholesterol                    | 47.00  | 61.00  | 14.00  | 26.000        |
| 11 | systolic_bp                        | 106.00 | 125.00 | 19.00  | 77.500        |
| 1  | alcohol_consumption_per_week       | 1.00   | 3.00   | 2.00   | -2.000        |
| 16 | ldl_cholesterol                    | 78.00  | 126.00 | 48.00  | 6.000         |
| 3  | diet_score                         | 4.80   | 7.20   | 2.40   | 1.200         |

16

|    |                           |        |        |       |        |
|----|---------------------------|--------|--------|-------|--------|
| 20 | insulin_level             | 5.09   | 12.45  | 7.36  | -5.950 |
| 14 | cholesterol_total         | 164.00 | 208.00 | 44.00 | 98.000 |
| 5  | screen_time_hours_per_day | 4.30   | 7.70   | 3.40  | -0.800 |
| 17 | triglycerides             | 91.00  | 151.00 | 60.00 | 1.000  |
| 10 | waist_to_hip_ratio        | 0.82   | 0.89   | 0.07  | 0.715  |
| 0  | age                       | 39.00  | 61.00  | 22.00 | 6.000  |
| 7  | hypertension_history      | 0.00   | 1.00   | 1.00  | -1.500 |

## 4 Data Cleaning

The cleaning process focused on preparing a reliable feature set while avoiding unnecessary alteration of the original observations.

First, there were no duplicated rows and no missing values requiring removal. Second, identifier-column analysis found no obvious patient-ID column requiring exclusion.

Third, potentially invalid measurements were explicitly identified using predefined domain ranges. These checks were used to assess data quality rather than

automatically discarding observations, since an unusual measurement may still carry useful predictive information.

Finally, two variables strongly associated with the diagnosis itself were excluded from the predictive representation. This step is important because including **target-derived information** could allow the classifier to exploit information that would not be legitimately available at prediction time.

After cleaning and feature selection, the resulting predictive data contain 28 original features.

## 5 Data Representation

The representation process follows:

CSV → DataFrame → Selected Feature Matrix → Encoded/Scaled Matrix  
→ Model Input.

For a single patient:

$$x_i \in \mathbb{R}^{28}$$

before preprocessing.

For the complete predictive dataset:

$$X \in \mathbb{R}^{100000 \times 28}.$$

The 28 original features consist of 22 numerical and 6 categorical variables.

### Numerical representation

Numerical features are passed through a pipeline containing:

Median Imputation → Standardization.

StandardScaler transforms the numerical features to a standardized scale.

## Categorical representation

Categorical variables are transformed using:

Most-Frequent Imputation → One-Hot Encoding.

For example, if a categorical variable contains three categories:

$$\{A, B, C\}$$

one-hot encoding represents them as:

$$A = [1, 0, 0]$$

$$B = [0, 1, 0]$$

$$C = [0, 0, 1].$$

This converts categorical information into numerical values without imposing an artificial ordinal relationship between categories. The implementation uses `handle_unknown="ignore"` so that previously unseen categorical values do not cause the deployment pipeline to fail.

## Final model representation

The preprocessing pipeline was fitted only on the training data and then reused to transform the validation and test sets. This produced:

$$X_{train} \in \mathbb{R}^{70000 \times 46}$$

$$X_{validation} \in \mathbb{R}^{15000 \times 46}$$

$$X_{test} \in \mathbb{R}^{15000 \times 46}.$$

Therefore:

$$d_{original} = 28$$

and:

$$d_{final} = 46.$$

The increase from 28 to 46 dimensions results from the expansion of categorical variables through one-hot encoding.

```
: # =====
# Separate candidate X and target y
# =====

TARGET = "diagnosed_diabetes"

# Candidate columns that should not be used as predictive inputs
IDENTIFIER_COLUMNS = [
    col for col in df.columns
    if (
        col.lower() == "id"
        or col.lower().endswith("_id")
        or "patient_id" in col.lower()
    )
]

# Target-derived variables that require leakage review
TARGET_DERIVED_COLUMNS = [
    col for col in [
        "diabetes_stage",
        "diabetes_risk_score"
    ]
    if col in df.columns
]

EXCLUDED_COLUMNS = list(
```

```

    dict.fromkeys(
        [TARGET]
        + IDENTIFIER_COLUMNS
        + TARGET_DERIVED_COLUMNS
    )
)

X = df.drop(
    columns=EXCLUDED_COLUMNS,
    errors="ignore"
)

y = df[TARGET].copy()

print("Excluded columns:")
for col in EXCLUDED_COLUMNS:
    print("-", col)

print("\nX shape:", X.shape)
print("y shape:", y.shape)

```

Excluded columns:

- diagnosed\_diabetes
- diabetes\_stage
- diabetes\_risk\_score

X shape: (100000, 28)

y shape: (100000,)

Original feature matrix:

X R<sup>^</sup>(100000 × 28)

One patient record:

|   |                                    |                              |                      |                 |              |                   |   |
|---|------------------------------------|------------------------------|----------------------|-----------------|--------------|-------------------|---|
|   | age                                | gender                       | ethnicity            | education_level | income_level | employment_status | \ |
| 0 | 58                                 | Male                         | Asian                | Highschool      | Lower-Middle | Employed          |   |
|   | smoking_status                     | alcohol_consumption_per_week | \                    |                 |              |                   |   |
| 0 | Never                              |                              | 0                    |                 |              |                   |   |
|   | physical_activity_minutes_per_week | diet_score                   | ...                  | diastolic_bp    | \            |                   |   |
| 0 | 215                                | 5.7                          | ...                  | 78              |              |                   |   |
|   | heart_rate                         | cholesterol_total            | hdl_cholesterol      | ldl_cholesterol | \            |                   |   |
| 0 | 68                                 | 239                          | 41                   | 160             |              |                   |   |
|   | triglycerides                      | glucose_fasting              | glucose_postprandial | insulin_level   | hba1c        |                   |   |
| 0 | 145                                | 136                          | 236                  | 6.36            | 8.18         |                   |   |

[1 rows x 28 columns]

## Final numerical representation

```
# =====  
# Inspect the final numerical representation  
# =====  
  
feature_names = (  
    preprocessor  
    .get_feature_names_out()  
)  
  
final_d = len(feature_names)  
  
print("Original feature dimension :", d_original)  
print("Final feature dimension     :", final_d)  
  
print(  
    f"\nFinal training representation:\n"  
    f"X_train      "  
    f"R^({X_train_processed.shape[0]} × {final_d})"  
)  
  
# Preview the first five transformed observations  
processed_preview = pd.DataFrame(  
    X_train_processed[:5],  
    columns=feature_names  
)  
  
display(  
    processed_preview  
)
```

|   | numeric__age | numeric__alcohol_consumption_per_week \ |
|---|--------------|-----------------------------------------|
| 0 | 0.057573     | -0.705971                               |
| 1 | 0.121663     | 1.405186                                |
| 2 | 1.339385     | -0.002252                               |
| 3 | -0.455152    | -0.002252                               |

63

|   |          |           |
|---|----------|-----------|
| 4 | 0.057573 | -0.705971 |
|---|----------|-----------|

|   | numeric__physical_activity_minutes_per_week | numeric__diet_score \ |
|---|---------------------------------------------|-----------------------|
| 0 | 0.010691                                    | 0.451912              |
| 1 | -0.604057                                   | 0.564192              |
| 2 | 0.909167                                    | 1.574706              |
| 3 | 1.866754                                    | -0.670881             |
| 4 | 0.329886                                    | -1.849814             |

|   | numeric__sleep_hours_per_day | numeric__screen_time_hours_per_day \ |
|---|------------------------------|--------------------------------------|
| 0 | 0.550392                     | -0.322613                            |
| 1 | -0.820324                    | 0.406408                             |
| 2 | 1.007298                     | 0.730417                             |
| 3 | -1.277229                    | -0.444117                            |
| 4 | -1.734135                    | 1.783447                             |

|   | numeric__family_history_diabetes | numeric__hypertension_history \ |
|---|----------------------------------|---------------------------------|
| 0 | 1.888283                         | 1.723319                        |
| 1 | -0.529582                        | -0.580276                       |
| 2 | -0.529582                        | 1.723319                        |
| 3 | -0.529582                        | -0.580276                       |
| 4 | -0.529582                        | -0.580276                       |

|   | numeric__cardiovascular_history | numeric__bmi ... \ |
|---|---------------------------------|--------------------|
| 0 | -0.293048                       | -0.280096 ...      |
| 1 | -0.293048                       | -0.419305 ...      |
| 2 | -0.293048                       | -0.502829 ...      |
| 3 | -0.293048                       | -0.530671 ...      |
| 4 | -0.293048                       | 0.360260 ...       |

|   | categorical__income_level_Lower-Middle | categorical__income_level_Middle \ |
|---|----------------------------------------|------------------------------------|
| 0 | 0.0                                    | 0.0                                |
| 1 | 0.0                                    | 0.0                                |
| 2 | 1.0                                    | 0.0                                |
| 3 | 1.0                                    | 0.0                                |
| 4 | 0.0                                    | 1.0                                |

|   | categorical__income_level_Upper-Middle \ |
|---|------------------------------------------|
| 0 | 1.0                                      |
| 1 | 1.0                                      |
| 2 | 0.0                                      |
| 3 | 0.0                                      |
| 4 | 0.0                                      |

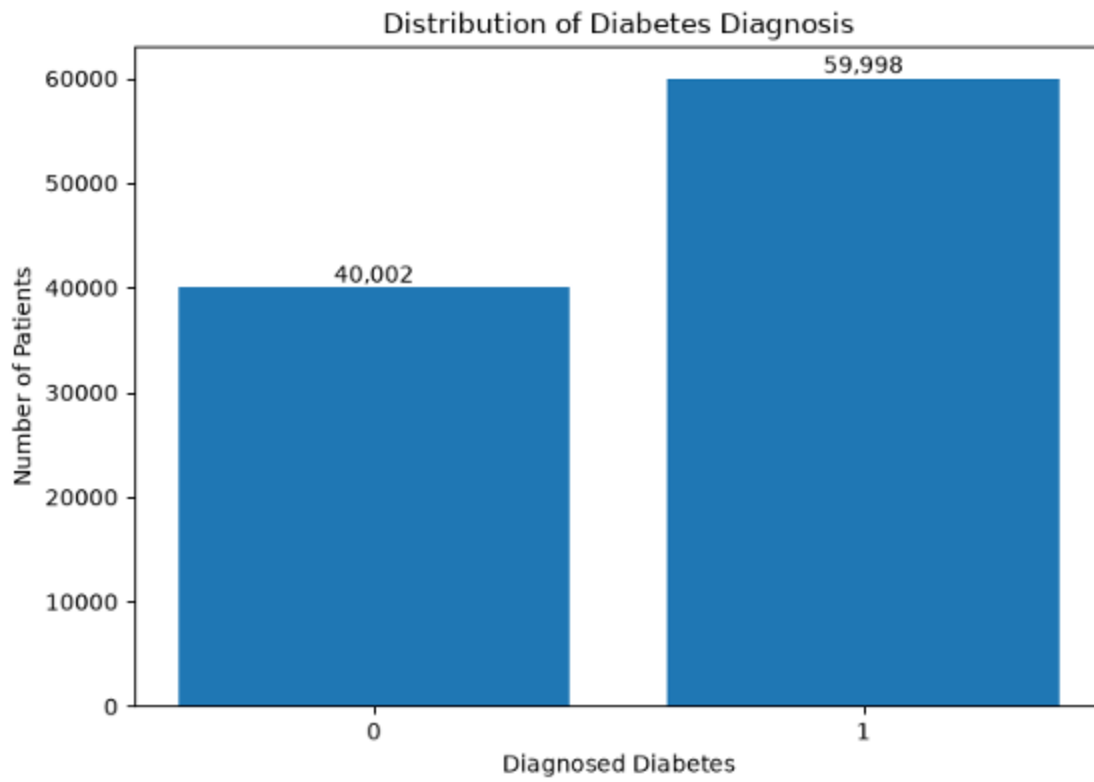
  

|   | categorical__employment_status_Employed \ |
|---|-------------------------------------------|
| 0 | 1.0                                       |
| 1 | 0.0                                       |
| 2 | 1.0                                       |

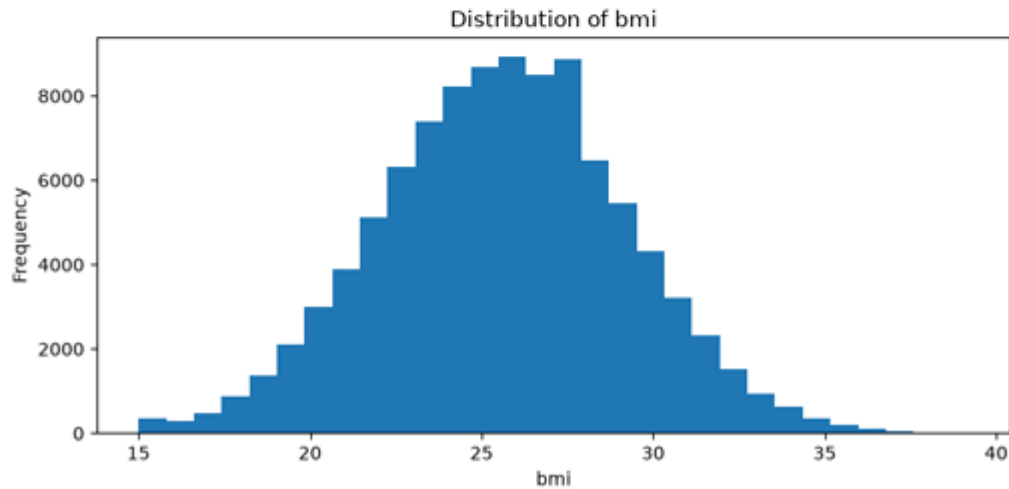
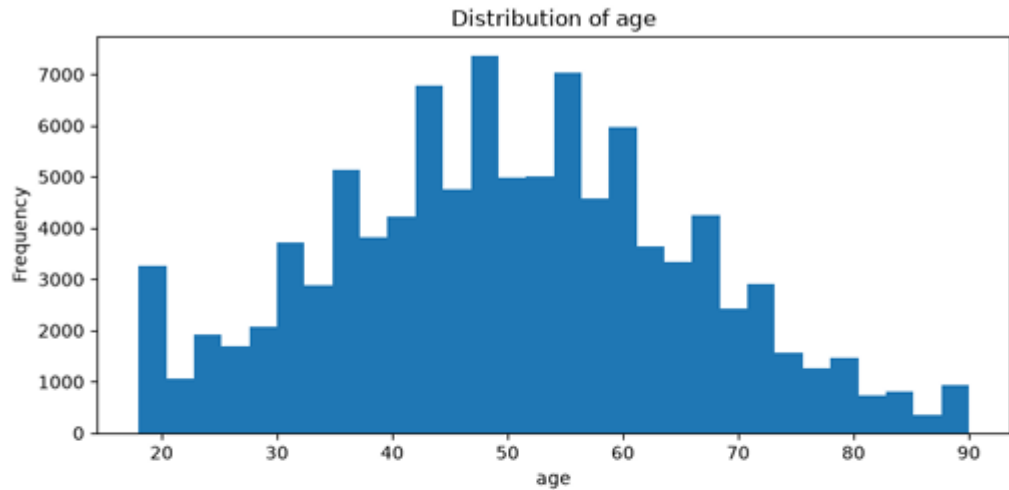
## 6. Exploratory Data Analysis

Exploratory analysis examined target distribution, numerical feature distributions, feature–target relationships, categorical distributions, pairwise relationships, and correlations.

### Target distributions

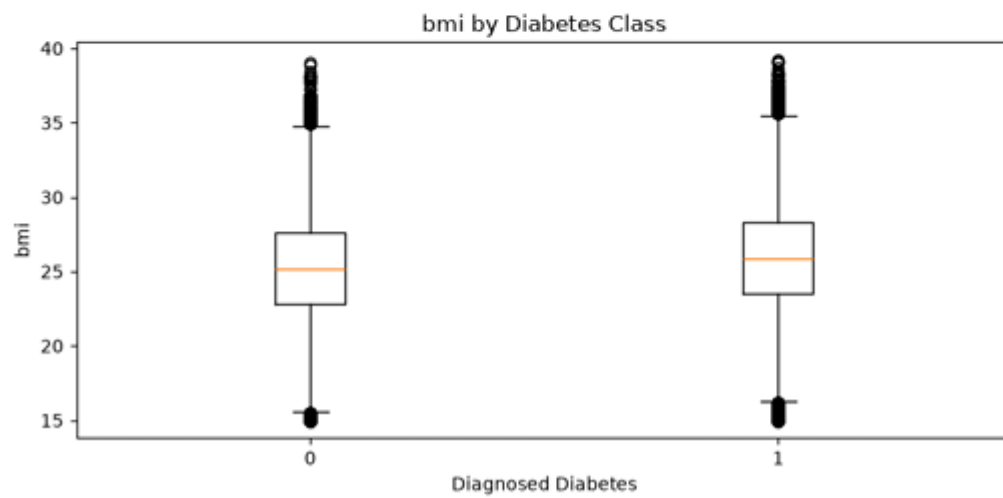
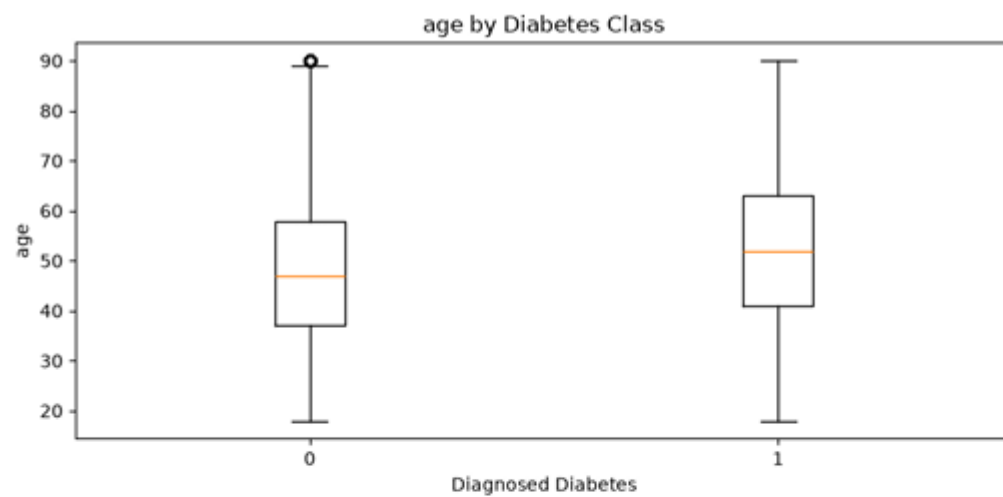


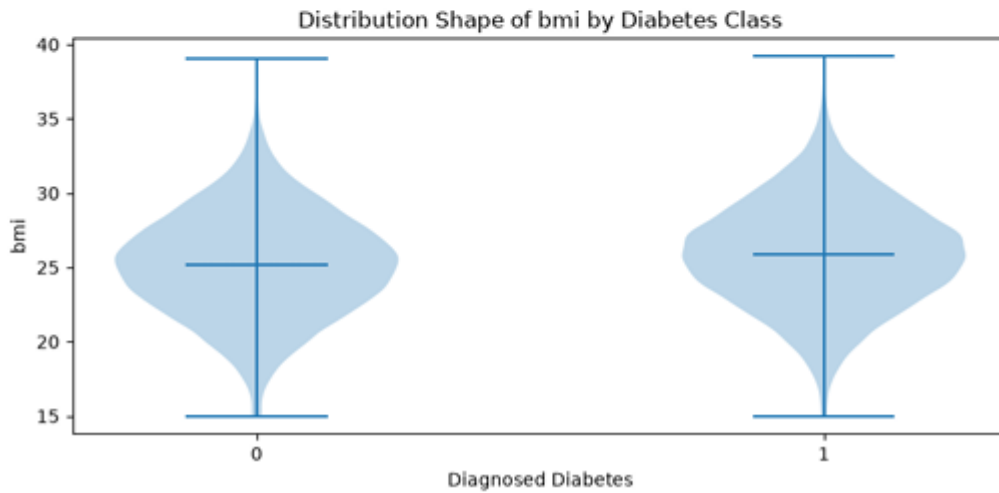
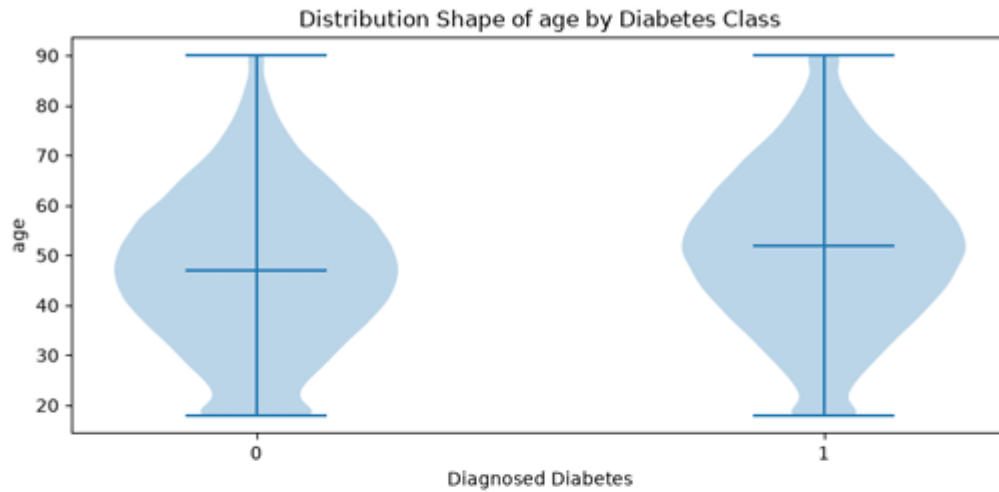
**Numerical feature distributions**



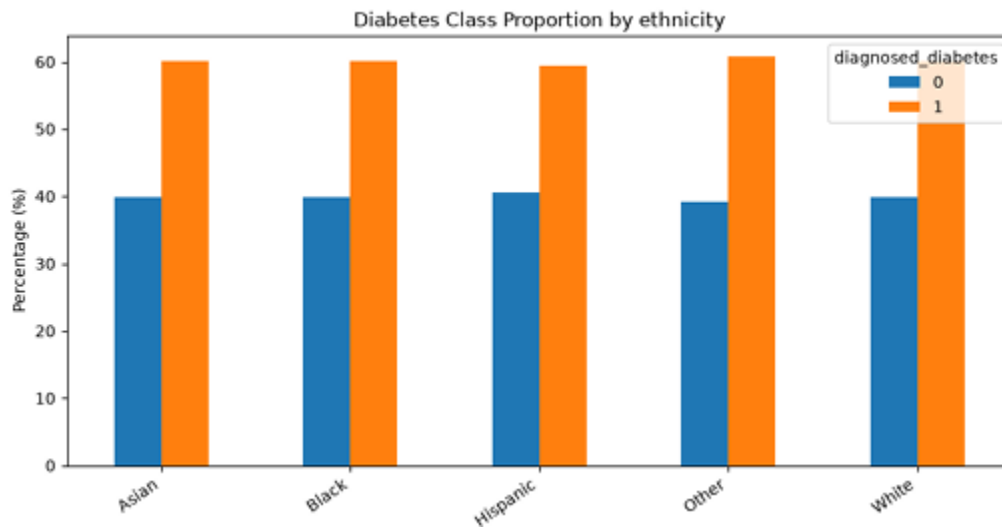
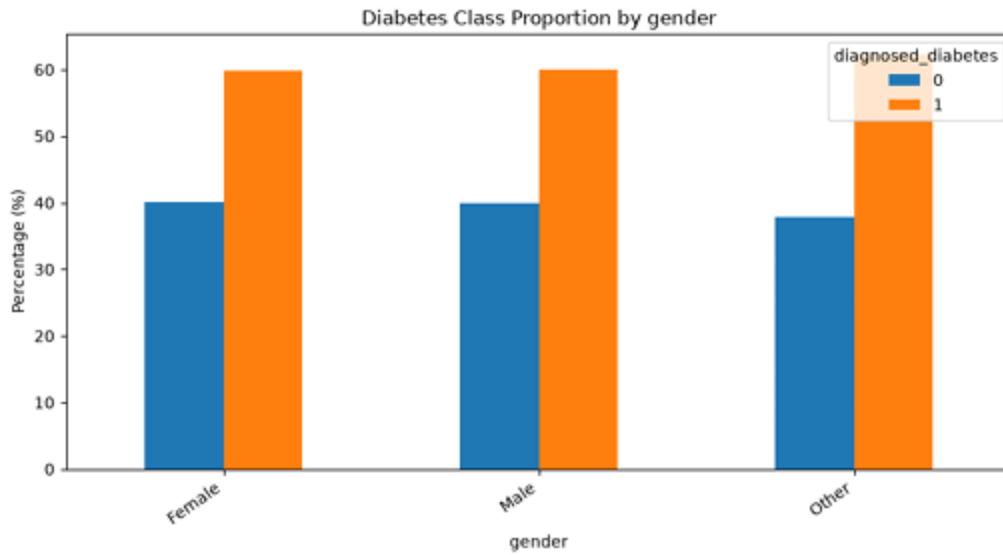
## Feature–target relationships

Numerical variables were also compared across diabetes classes using box plots and violin plots.



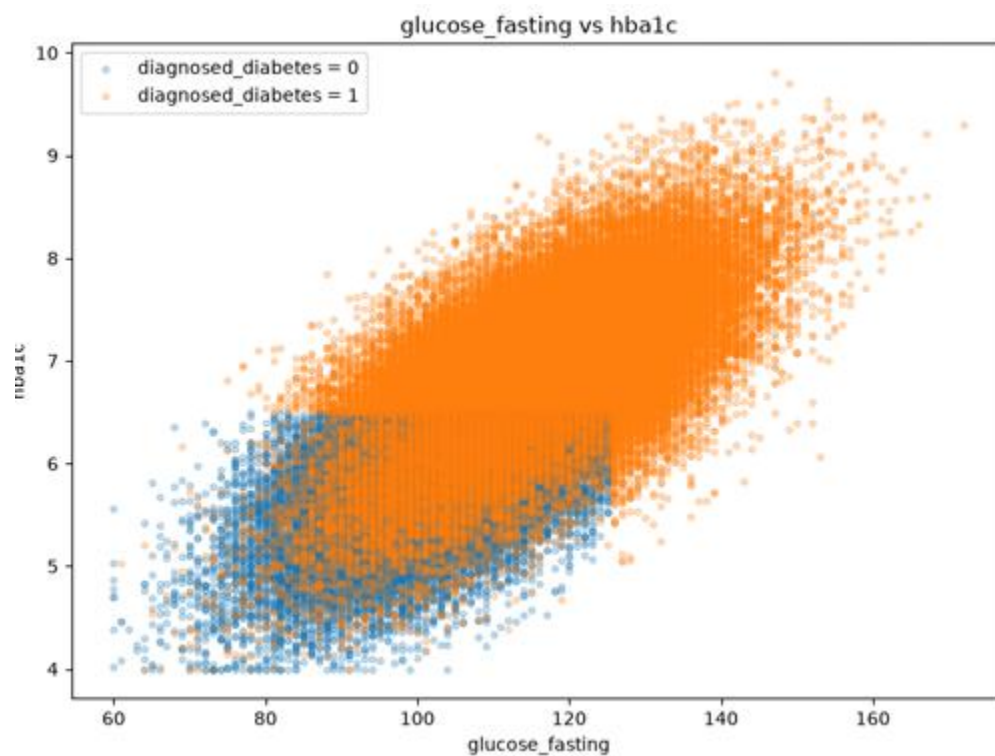
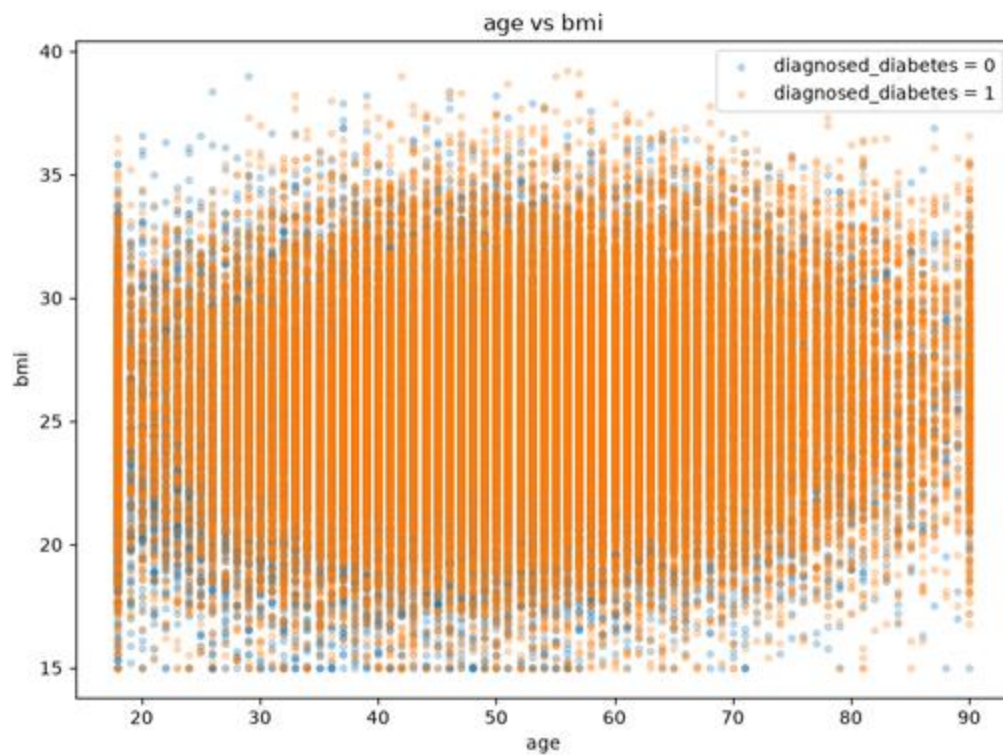


**Categorical features across diabetes classes**



## Relationships between important features

The notebook additionally examines relationships such as:



Overall, the EDA indicates that the dataset contains substantial information relevant to diabetes classification, particularly across clinical and metabolic measurements.

## 7 Model Development

The data were divided into training, validation, and test subsets using a **70/15/15 stratified split**:

| Dataset    | Rows       |
|------------|------------|
| Training   | 70,00<br>0 |
| Validation | 15,00<br>0 |
| Test       | 15,00<br>0 |

## Dataset

## Rows

```
[51]: # =====  
# 70/15/15 stratified split  
# =====  
  
from sklearn.model_selection import train_test_split  
  
# First split:  
# 70% train  
# 30% temporary set  
X_train, X_temp, y_train, y_temp = train_test_split(  
    X,  
    y,  
    test_size=0.30,  
    stratify=y,  
    random_state=RANDOM_SEED  
)  
  
# Second split:  
# 15% validation  
# 15% test  
X_val, X_test, y_val, y_test = train_test_split(  
    X_temp,  
    y_temp,  
    test_size=0.50,  
    stratify=y_temp,  
    random_state=RANDOM_SEED  
)  
  
print("Training set   :", X_train.shape)  
print("Validation set :", X_val.shape)  
print("Test set       :", X_test.shape)
```

```
Training set   : (70000, 28)  
Validation set : (15000, 28)  
Test set      : (15000, 28)
```

```
# =====  
# Fit preprocessing on training data only  
# =====  
  
preprocessor.fit(X_train)  
  
# Transform each dataset using the same fitted preprocessor  
X_train_processed = preprocessor.transform(X_train)  
X_val_processed = preprocessor.transform(X_val)  
X_test_processed = preprocessor.transform(X_test)
```

The preprocessing object was fitted only on the training set. The validation set was used for model comparison and hyperparameter experiments, while the test set remained untouched until the final evaluation.

## Baseline model

```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

baseline_model = DummyClassifier(
    strategy="prior"
)

baseline_model.fit(
    X_train_processed,
    y_train
)

baseline_pred = baseline_model.predict(
    X_val_processed
)

baseline_score = (
    baseline_model
    .predict_proba(X_val_processed)[: , 1]
)
```

A DummyClassifier was used as a non-learning baseline. Its validation results were approximately:

|   | Model          | Accuracy | Precision | Recall | F1   | ROC-AUC |
|---|----------------|----------|-----------|--------|------|---------|
| 0 | Dummy Baseline | 0.6      | 0.6       | 1.0    | 0.75 | 0.5     |

The baseline provides a reference point against which the learning-based classifiers can be evaluated.

## **Classification models**

Five classifiers were trained:

- Logistic Regression
- KNN
- Decision Tree
- Random Forest
- SVM

The models represent different learning assumptions, including linear, distance-based, tree-based, ensemble, and kernel-based approaches.

```

models = {
    "Logistic Regression": LogisticRegression(
        max_iter=1000,
        class_weight="balanced",
        random_state=RANDOM_SEED
    ),

    "KNN": KNeighborsClassifier(
        n_neighbors=15,
        weights="distance",
        n_jobs=-1
    ),

    "Decision Tree": DecisionTreeClassifier(
        max_depth=8,
        min_samples_leaf=20,
        class_weight="balanced",
        random_state=RANDOM_SEED
    ),

    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        max_depth=12,
        min_samples_leaf=5,
        class_weight="balanced",
        n_jobs=-1,
        random_state=RANDOM_SEED
    ),

    "SVM": SVC(
        C=1.0,
        kernel="rbf",
        probability=True,
        class_weight="balanced",
        random_state=RANDOM_SEED
    )
}

```

```

for model_name, model in models.items():

    print(
        f"\nTraining: {model_name}"
    )

    start_time = time.perf_counter()

    model.fit(
        X_train_processed,
        y_train

```

70

```

)

    training_time = (
        time.perf_counter()
        - start_time
    )

    predictions, scores, metrics = (
        evaluate_classifier(
            model,
            X_val_processed,
            y_val
        )
    )

```

## Hyperparameter experiments

Three focused experiments were conducted.

### Experiment 1 — Logistic Regression

*How does the regularization strength  $C$  affect validation performance?*

The values:

$$C \in \{0.01, 0.1, 1, 10\}$$

were evaluated. The best F1 result occurred at:

$$C = 1$$

with  $F1 = 0.904574$ .

|   | C     | Accuracy | Precision | Recall   | F1       | ROC-AUC  | Training Time (s) |
|---|-------|----------|-----------|----------|----------|----------|-------------------|
| 2 | 1.00  | 0.888867 | 0.932932  | 0.877889 | 0.904574 | 0.934740 | 0.1817            |
| 3 | 10.00 | 0.888800 | 0.932822  | 0.877889 | 0.904522 | 0.934744 | 0.2079            |
| 1 | 0.10  | 0.888600 | 0.932696  | 0.877667 | 0.904345 | 0.934716 | 0.1745            |
| 0 | 0.01  | 0.887600 | 0.931963  | 0.876667 | 0.903470 | 0.934281 | 0.2499            |

## Experiment 2 — Random Forest

*Does increasing the number of trees improve predictive performance enough to justify the additional computational cost?*

The values:

$$n_{estimators} \in \{50, 100, 200, 300\}$$

produced identical accuracy, precision, recall, and F1 values, while training time increased from 0.9076 seconds at 50 trees to 7.9661 seconds at 300 trees. ROC-AUC improved only marginally.

|   | n_estimators | Accuracy | Precision | Recall   | F1       | ROC-AUC  | \ |
|---|--------------|----------|-----------|----------|----------|----------|---|
| 0 | 50           | 0.920067 | 1.0       | 0.866778 | 0.928635 | 0.942401 |   |
| 1 | 100          | 0.920067 | 1.0       | 0.866778 | 0.928635 | 0.942701 |   |
| 2 | 200          | 0.920067 | 1.0       | 0.866778 | 0.928635 | 0.942932 |   |
| 3 | 300          | 0.920067 | 1.0       | 0.866778 | 0.928635 | 0.943119 |   |

## Experiment 3 — Decision Tree

*How does tree depth affect the balance between model complexity and predictive performance?*

The evaluated values were:

$$max\_depth \in \{3, 5, 8, 12, 16\}.$$

The shallow trees at depths 3 and 5 produced the highest F1 of 0.928635, while deeper trees did not improve F1 and increased training time.

These experiments show that increasing model complexity does not automatically produce a meaningful performance improvement.

|   | max_depth | Accuracy | Precision | Recall   | F1       | ROC-AUC  | \ |
|---|-----------|----------|-----------|----------|----------|----------|---|
| 0 | 3         | 0.920067 | 1.000000  | 0.866778 | 0.928635 | 0.938618 |   |
| 1 | 5         | 0.920067 | 1.000000  | 0.866778 | 0.928635 | 0.943473 |   |
| 2 | 8         | 0.919533 | 0.998337  | 0.867333 | 0.928236 | 0.944031 |   |
| 3 | 12        | 0.918267 | 0.995285  | 0.867889 | 0.927232 | 0.941733 |   |
| 4 | 16        | 0.916467 | 0.990379  | 0.869222 | 0.925854 | 0.939768 |   |

## 8 Evaluation

The models were evaluated using Accuracy, Precision, Recall, F1-score, ROC-AUC, and training time.

### Model comparison

|   | Model               | Accuracy | Precision | Recall   | F1       | ROC-AUC  | \ |
|---|---------------------|----------|-----------|----------|----------|----------|---|
| 0 | Random Forest       | 0.920067 | 1.000000  | 0.866778 | 0.928635 | 0.942932 |   |
| 1 | Decision Tree       | 0.919533 | 0.998337  | 0.867333 | 0.928236 | 0.944031 |   |
| 2 | SVM                 | 0.893800 | 0.972445  | 0.847000 | 0.905398 | 0.933993 |   |
| 3 | Logistic Regression | 0.888867 | 0.932932  | 0.877889 | 0.904574 | 0.934740 |   |
| 4 | KNN                 | 0.835067 | 0.867372  | 0.856000 | 0.861649 | 0.908827 |   |

Random Forest achieved the highest F1-score and accuracy, while Decision Tree achieved slightly higher recall and ROC-AUC. SVM delivered good predictive performance but required substantially more computational time.

### Evaluation metrics

For this medical classification task, Recall is especially important because a false negative corresponds to a diabetes-positive patient that the classifier fails to identify. However, maximizing Recall alone is undesirable because excessive false positives reduce Precision.

F1-score therefore provides a useful balance between Precision and Recall, while ROC-AUC provides an additional view of the classifier's class-separation capability.

## 9 Model Selection

The deployment candidate was selected using a combined ranking of:

- F1-score;
- Recall;
- ROC-AUC.

The ranking was:

|   | Model         | Accuracy | Precision | Recall   | F1       | ROC-AUC  | \ |
|---|---------------|----------|-----------|----------|----------|----------|---|
| 0 | Decision Tree | 0.919533 | 0.998337  | 0.867333 | 0.928236 | 0.944031 |   |

94

|   |                     |          |          |          |          |          |  |
|---|---------------------|----------|----------|----------|----------|----------|--|
| 1 | Random Forest       | 0.920067 | 1.000000 | 0.866778 | 0.928635 | 0.942932 |  |
| 2 | Logistic Regression | 0.888867 | 0.932932 | 0.877889 | 0.904574 | 0.934740 |  |
| 3 | SVM                 | 0.893800 | 0.972445 | 0.847000 | 0.905398 | 0.933993 |  |
| 4 | KNN                 | 0.835067 | 0.867372 | 0.856000 | 0.861649 | 0.908827 |  |

|   | Training Time (s) | F1_rank | Recall_rank | ROC_AUC_rank | Combined_rank |
|---|-------------------|---------|-------------|--------------|---------------|
| 0 | 0.4287            | 2.0     | 2.0         | 1.0          | 5.0           |
| 1 | 3.1413            | 1.0     | 3.0         | 2.0          | 6.0           |
| 2 | 0.2092            | 4.0     | 1.0         | 3.0          | 8.0           |
| 3 | 1571.5295         | 3.0     | 5.0         | 4.0          | 12.0          |
| 4 | 0.0059            | 5.0     | 4.0         | 5.0          | 14.0          |

The **Decision Tree** was therefore selected as the final deployment model.

Random Forest obtained a marginally higher F1-score ,but the Decision Tree achieved a better combined ranking because it provided competitive F1, higher Recall and ROC-AUC, and considerably lower training cost. Its simpler structure also provides better interpretability and is more convenient for deployment.

The final model configuration used for persistence is:

DecisionTreeClassifier(

```
max_depth=8,  
min_samples_leaf=20,  
class_weight="balanced",  
random_state=42  
)
```

The final test performance confirms that the selected model generalizes closely to the held-out observations, with a test F1 of 0.928758 and ROC-AUC of 0.942061.

```

final_test_pred, final_test_scores, final_test_metrics = (
    evaluate_classifier(
        selected_model,
        X_test_processed,
        y_test
    )
)

final_test_results = pd.DataFrame([
    {
        "Model": selected_model_name,
        **final_test_metrics
    }
])

display(
    final_test_results
)

```

|   | Model         | Accuracy | Precision | Recall   | F1       | ROC-AUC  |
|---|---------------|----------|-----------|----------|----------|----------|
| 0 | Decision Tree | 0.92     | 0.997068  | 0.869208 | 0.928758 | 0.942061 |

```

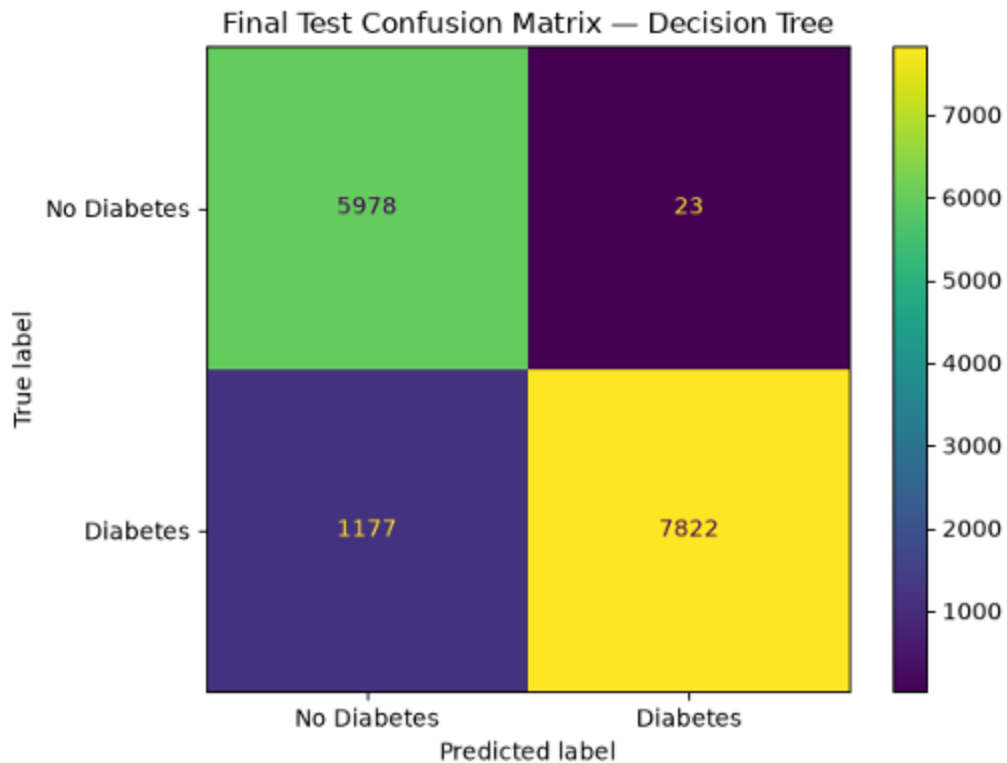
# =====
# Final test confusion matrix
# =====

ConfusionMatrixDisplay.from_predictions(
    y_test,
    final_test_pred,
    display_labels=[
        "No Diabetes",
        "Diabetes"
    ]
)

plt.title(
    f"Final Test Confusion Matrix - "
    f"{selected_model_name}"
)

plt.tight_layout()
plt.show()

```



## 10 Deployment

The selected Decision Tree was combined with the fitted preprocessing pipeline into a complete machine-learning pipeline.

The pipeline was trained on the training data and persisted as:

model/diabetes\_model\_pipeline.joblib

The same artifact contains preprocessing and the final classifier, ensuring consistency in:

- imputation;
- categorical encoding;
- scaling;
- feature ordering;
- model inference.

Reproducibility metadata were additionally stored in:

model/metadata.json

including dataset information, selected model, random seed, environment information, split sizes, and feature dimensions.

## **Web Service**

The prediction service is implemented using Flask.

The main prediction endpoint is:

POST /diabetes/v1/predict

The inference workflow is:

Patient Input → Validation → Preprocessing → Decision Tree → Prediction.

The service returns the predicted diabetes class together with probability and confidence information.

The Web interface provides:

- patient information input;
- numerical and categorical controls;
- sample data for testing;
- prediction submission;
- diagnosis;
- confidence;
- diabetes probability;
- no-diabetes probability;
- selected model information.

# Diabetes Prediction

Machine Learning Based Health Assessment

## Patient Information

Enter the patient information below. Sample values can be loaded automatically for testing.

Age

Alcohol Consumption Per Week

Physical Activity Minutes Per Week

Diet Score

Sleep Hours Per Day

Screen Time Hours Per Day

Family History Diabetes

Hypertension History

Cardiovascular History

Bmi

Waist To Hip Ratio

Systolic Bp

Diastolic Bp

Heart Rate

Cholesterol Total

28.5

0.9

125

Diastolic Bp

80

Heart Rate

72

Cholesterol Total

190

Hdl Cholesterol

55

Ldl Cholesterol

110

Triglycerides

140

Glucose Fasting

105

Glucose Postprandial

145

Insulin Level

12

Hba1c

5.8

Gender

Male

Ethnicity

White

Education Level

Select Education Level

Income Level

Middle

Employment Status

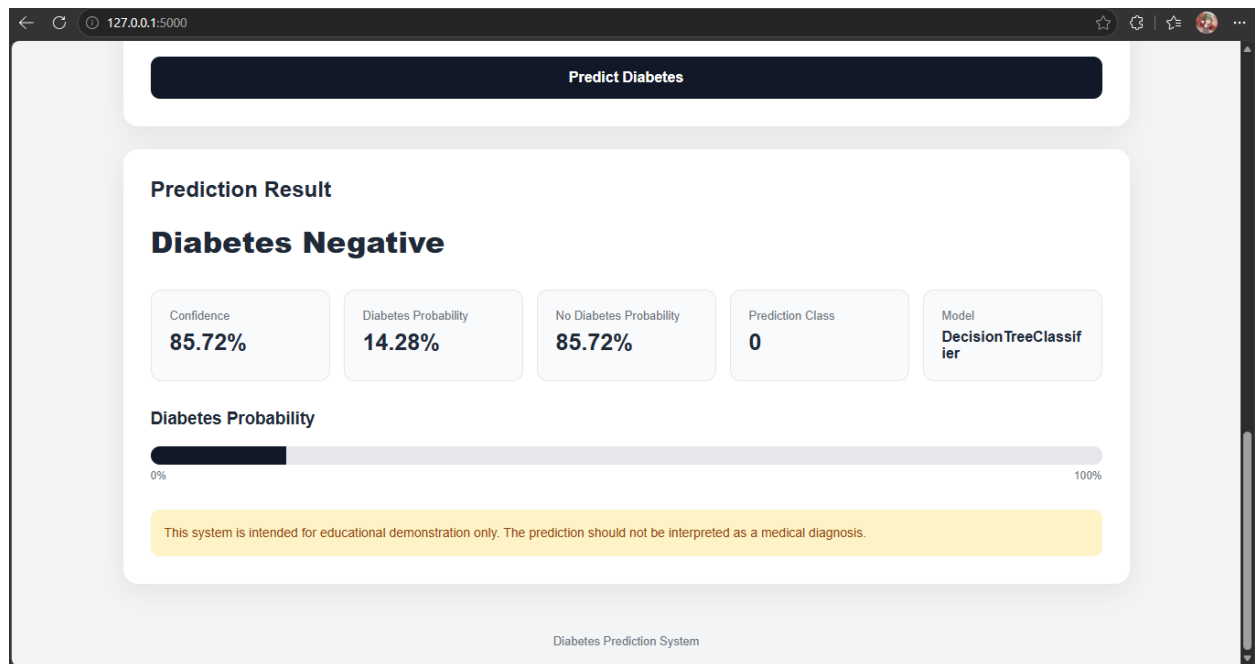
Employed

Smoking Status

Never

Fill Sample Data

Predict Diabetes



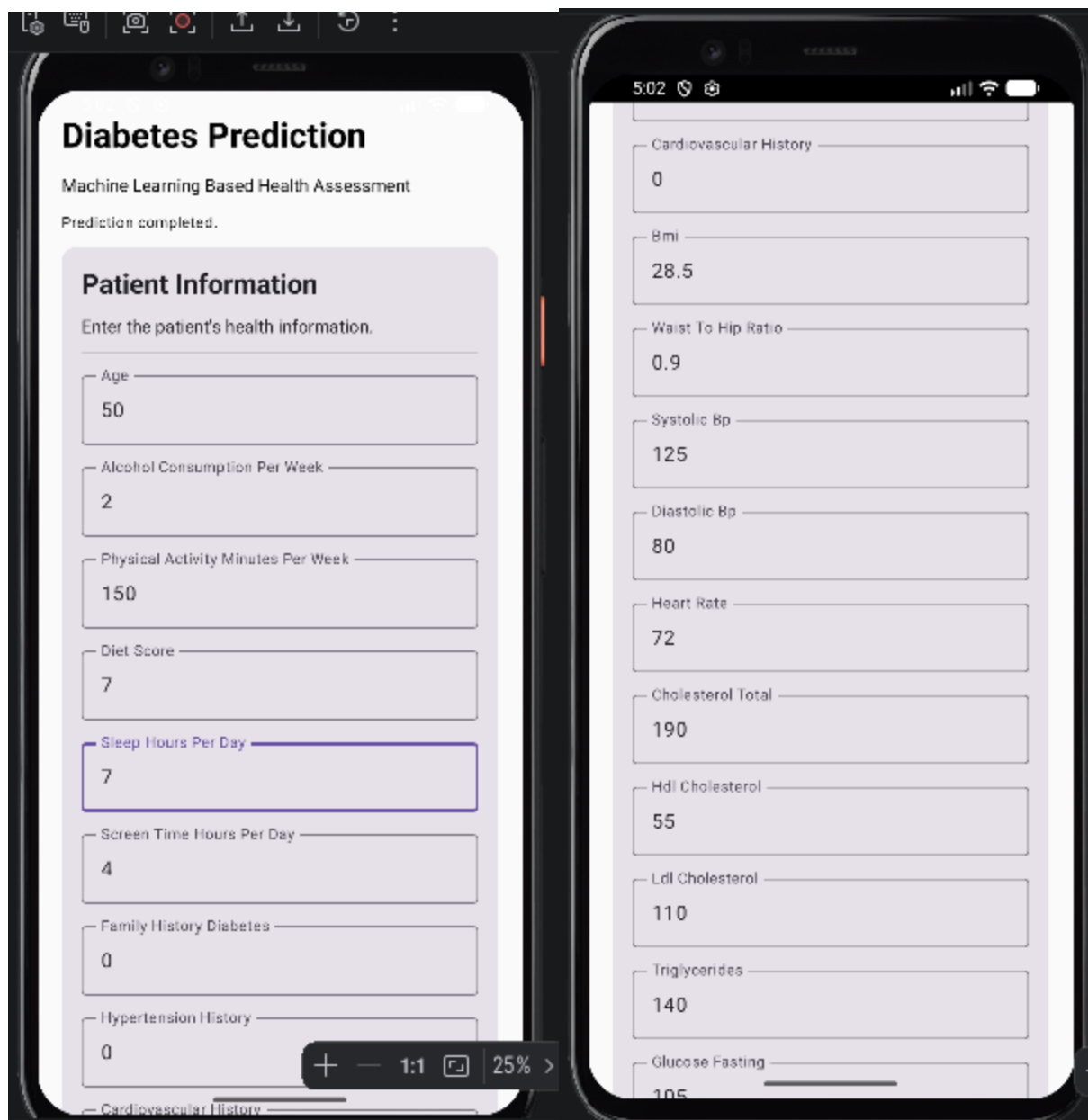
## Mobile Application

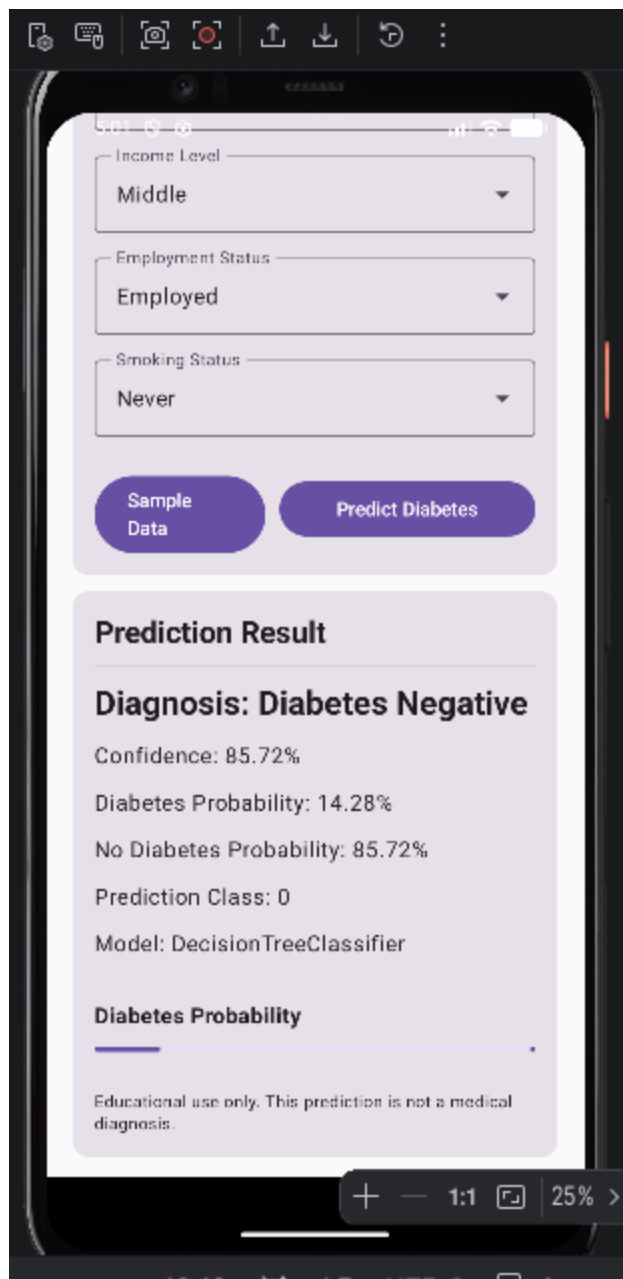
The mobile client was implemented using **Android Studio, Kotlin, and Jetpack Compose**. It communicates with the same Flask prediction service rather than embedding the trained model inside the Android application.

The architecture is:

Mobile UI → REST API → Saved Preprocessing → Saved Model → Prediction  
→ Mobile UI.

The mobile interface dynamically obtains the model schema, provides patient-input controls, supports sample data, and displays the returned diagnosis and probability information.





that the saved preprocessing/model pipeline can be reused for inference independently of the training process.

## B. Application 2 — House Price Prediction

### 1 Problem Description

House price prediction is formulated as a supervised regression problem. The objective is to estimate the historical price of a property from information that describes the property and its geographical location.

The problem is defined as:

$$X = \text{house characteristics}$$
$$y = \text{house price}$$

where each row represents one property transaction record and the target variable is `history_price`.

The prediction problem is challenging because house prices are influenced by several factors at the same time. Property size and room count provide information about the physical characteristics of a property, while latitude, longitude, outcode and related location attributes capture geographical differences in the London housing market. The dataset also contains properties with very different price levels, including a relatively small number of very high-value transactions. This creates a strongly skewed target distribution and makes large prediction errors particularly important.

The system is designed around the following pipeline:

Raw Data → Cleaning → Representation → Regression Model → Prediction

The objective is not only to obtain a strong predictive model, but also to produce a representation that can be consistently reused during deployment.

### 2 Dataset

The dataset used in this application is the **London House Price Data** available from Kaggle. The dataset contains property characteristics, location information, transaction history and price-related variables.

The local notebook loads 418,201 observations with 28 original columns.

The target variable is:

history\_price

The main predictive variables used in the current experiment include:

| <b>Feature</b>      | <b>Type</b> | <b>Description</b>          |
|---------------------|-------------|-----------------------------|
| latitude            | Numerical   | Geographic latitude         |
| longitude           | Numerical   | Geographic longitude        |
| outcode             | Categorical | Broad postcode area         |
| bathrooms           | Numerical   | Number of bathrooms         |
| bedrooms            | Numerical   | Number of bedrooms          |
| floorAreaSqM        | Numerical   | Floor area in square metres |
| livingRooms         | Numerical   | Number of living rooms      |
| tenure              | Categorical | Property tenure             |
| propertyType        | Categorical | Property type               |
| currentEnergyRating | Categorical | Current energy rating       |

The original dataset also contains detailed address information, postcode, rent estimates, sale estimates and historical variables. The raw dataset contains 137,760 distinct full addresses and 48,392 distinct postcodes, showing that these fields have very high cardinality.

For the prediction problem, location remains an important part of the representation. The structured geographical variables latitude, longitude and outcode are therefore retained rather than treating location as irrelevant information.

```
print(
    "Number of observations:",
    len(df)
)

print(
    "Number of columns:",
    df.shape[1]
)

print(
    "Dataset shape:",
    df.shape
)
```

Number of observations: 418201  
Number of columns: 28  
Dataset shape: (418201, 28)

### 3 Data Understanding and Cleaning

Before model development, the dataset was examined to identify missing values, duplicate records, invalid observations, outliers and variable types.

Column data types:

|                                           | dtype   |
|-------------------------------------------|---------|
| fullAddress                               | str     |
| postcode                                  | str     |
| country                                   | str     |
| outcode                                   | str     |
| latitude                                  | float64 |
| longitude                                 | float64 |
| bathrooms                                 | float64 |
| bedrooms                                  | float64 |
| floorAreaSqM                              | float64 |
| livingRooms                               | float64 |
| tenure                                    | str     |
| propertyType                              | str     |
| currentEnergyRating                       | str     |
| rentEstimate_lowerPrice                   | float64 |
| rentEstimate_currentPrice                 | float64 |
| rentEstimate_upperPrice                   | float64 |
| saleEstimate_lowerPrice                   | float64 |
| saleEstimate_currentPrice                 | float64 |
| saleEstimate_upperPrice                   | float64 |
| saleEstimate_confidenceLevel              | str     |
| saleEstimate_ingestedAt                   | str     |
| saleEstimate_valueChange.numericChange    | float64 |
| saleEstimate_valueChange.percentageChange | float64 |
| saleEstimate_valueChange.saleDate         | str     |
| history_date                              | str     |
| history_price                             | int64   |
| history_percentageChange                  | float64 |
| history_numericChange                     | float64 |

## Missing values

Missing values are present in several property attributes. The largest missing proportions include currentEnergyRating, bathrooms, livingRooms, bedrooms, floorAreaSqM and tenure. The historical change variables also contain substantial missingness.

```
[6]: # =====  
# 6. Missing-value analysis  
# =====  
  
missing_summary = (  
    df.isna()  
    .sum()  
    .to_frame(  
        "Missing Count"  
    )  
)  
  
missing_summary["Percentage"] = (  
    missing_summary["Missing Count"]  
    / len(df)  
    * 100  
)  
  
missing_summary = (  
    missing_summary  
    .query(  
        "`Missing Count` > 0"  
    )  
    .sort_values(  
        "Missing Count",  
        ascending=False  
    )  
)
```

```

    )
)

if missing_summary.empty:

    print(
        "No missing values were found."
    )

else:

    display(
        missing_summary
    )

```

|                                           | Missing Count | Percentage |
|-------------------------------------------|---------------|------------|
| history_percentageChange                  | 180219        | 43.093871  |
| history_numericChange                     | 180219        | 43.093871  |
| currentEnergyRating                       | 84526         | 20.211812  |
| bathrooms                                 | 77931         | 18.634819  |
| livingRooms                               | 60500         | 14.466728  |
| bedrooms                                  | 40536         | 9.692947   |
| floorAreaSqM                              | 25210         | 6.028202   |
| tenure                                    | 11512         | 2.752743   |
| rentEstimate_upperPrice                   | 1741          | 0.416307   |
| rentEstimate_currentPrice                 | 1741          | 0.416307   |
| rentEstimate_lowerPrice                   | 1741          | 0.416307   |
| propertyType                              | 1136          | 0.271640   |
| saleEstimate_currentPrice                 | 640           | 0.153036   |
| saleEstimate_lowerPrice                   | 640           | 0.153036   |
| saleEstimate_upperPrice                   | 640           | 0.153036   |
| saleEstimate_confidenceLevel              | 640           | 0.153036   |
| saleEstimate_valueChange.numericChange    | 640           | 0.153036   |
| saleEstimate_ingestedAt                   | 640           | 0.153036   |
| saleEstimate_valueChange.saleDate         | 640           | 0.153036   |
| saleEstimate_valueChange.percentageChange | 640           | 0.153036   |

Instead of deleting all rows containing missing values, the model pipeline handles missing values according to feature type.

=> Numerical variables are imputed using their median values, while categorical variables are filled using the most frequent category. This approach preserves a much larger proportion of the dataset while keeping the transformation consistent between training and inference.

## Duplicate records

The dataset contains seven exact duplicate rows.

```
duplicate_count = (  
    df.duplicated()  
    .sum()  
)  
  
print(  
    "Duplicate rows:",  
    duplicate_count  
)  
  
rows_before = len(df)  
  
df = (  
    df  
    .drop_duplicates()  
    .reset_index(  
        drop=True  
    )  
)  
  
print(  
    "Rows removed:",  
    rows_before - len(df)  
)  
  
print(  
    "Rows after duplicate removal:",  
    len(df)  
)
```

```
Duplicate rows: 7  
Rows removed: 7  
Rows after duplicate removal: 418194
```

These duplicates are removed before model development because repeated copies of exactly the same observation can give those records a disproportionate influence during training.

## Invalid values

Invalid values are treated differently from statistical outliers. A high house price may be unusual, but it is not necessarily incorrect. In contrast, a non-positive house price or a negative room count would violate the basic meaning of the variable.

```
[8]: # =====  
# 8. Invalid-value analysis  
# =====  
  
invalid_checks = {}  
  
if TARGET in df.columns:  
    invalid_checks[TARGET] = (  
        df[TARGET].notna()  
        &  
        (df[TARGET] <= 0)  
    )  
  
for column in [  
    "floorAreaSqM",  
    "bedrooms",  
    "bathrooms",  
    "livingRooms"  
]:  
    if column in df.columns:  
        invalid_checks[column] = (  
            df[column].notna()  
            &  
            (df[column] < 0)  
        )  
  
invalid_results = []  
  
for column, mask in invalid_checks.items():
```

```

invalid_results.append({
    "Feature":
        column,

    "Invalid Count":
        int(mask.sum()),

    "Percentage":
        round(
            mask.mean() * 100,
            3
        )
})

invalid_df = pd.DataFrame(
    invalid_results
)

display(
    invalid_df
)

```

|   | Feature       | Invalid Count | Percentage |
|---|---------------|---------------|------------|
| 0 | history_price | 0             | 0.0        |
| 1 | floorAreaSqM  | 0             | 0.0        |
| 2 | bedrooms      | 0             | 0.0        |
| 3 | bathrooms     | 0             | 0.0        |
| 4 | livingRooms   | 0             | 0.0        |

The notebook checks for non-positive prices and negative values in the main property measurements. No such invalid records were identified, so no observations were removed for this reason.

## Outliers

The target variable is highly right-skewed. The mean historical price is approximately £639,584, while the median is £400,000 and the maximum is above £105 million.

|   | Feature       | Outlier Count | Percentage |
|---|---------------|---------------|------------|
| 0 | history_price | 34699         | 8.30       |
| 1 | floorAreaSqM  | 23355         | 5.94       |
| 2 | bedrooms      | 22333         | 5.91       |
| 3 | bathrooms     | 6085          | 1.79       |
| 4 | livingRooms   | 82907         | 23.18      |

This difference indicates a long upper tail. The extreme values are therefore important for model evaluation even though they should not be removed automatically. In a real property market, expensive properties are legitimate observations and removing them simply because they are statistically extreme could distort the task.

## Feature-selection consideration

The dataset contains several fields that are not suitable as direct model inputs for the intended prediction scenario. Detailed address strings are highly specific, while current estimate fields describe other valuation outputs rather than the core property characteristics used by the application.

```
Numerical features: 8
['latitude', 'longitude', 'bathrooms', 'bedrooms', 'floorAreaSqM',
'livingRooms', 'history_percentageChange', 'history_numericChange']

Categorical features: 6
['country', 'outcode', 'tenure', 'propertyType', 'currentEnergyRating',
'history_date']

Excluded columns:
['fullAddress', 'history_price', 'postcode', 'rentEstimate_currentPrice',
'rentEstimate_lowerPrice', 'rentEstimate_upperPrice',
'saleEstimate_confidenceLevel', 'saleEstimate_currentPrice',
'saleEstimate_ingestedAt', 'saleEstimate_lowerPrice', 'saleEstimate_upperPrice',
'saleEstimate_valueChange.numericChange',
'saleEstimate_valueChange.percentageChange',
'saleEstimate_valueChange.saleDate']
```

## 4 Representation

The representation is constructed through four stages:

Raw CSV → DataFrame → Clean Feature Set → Encoded and Scaled Model Input

For one property:

$$x_i = [x_{i1}, x_{i2}, \dots, x_{id}]^T$$

For the full dataset:

$$X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{bmatrix} \in \mathbb{R}^{N \times d}$$

and the target is:

$$y \in \mathbb{R}^N$$

The original data contains 418,201 observations and 28 columns. After duplicate removal, 418,194 observations remain for the modelling workflow.

In the current notebook, the engineered representation contains 16 features before encoding. After preprocessing, the training representation contains 205 numerical dimensions.

### **Numerical representation**

Numerical variables are converted into a common scale using standardization:

$$z = \frac{x - \mu}{\sigma}$$

where the mean and standard deviation are learned from the training data.

```
numeric_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="median"
        )
    ),
    (
        "scaler",
        StandardScaler()
    )
])
```

## Categorical representation

Categorical variables are converted into binary numerical features using one-hot encoding. For example, a property type with several possible categories is represented by a corresponding set of indicator variables.

Unknown categories are ignored during transformation so that a previously unseen category does not cause the deployed service to fail.

```
categorical_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="most_frequent"
        )
    ),
    (
```

47

```
        "onehot",
        OneHotEncoder(
            handle_unknown="ignore",
            min_frequency=20,
            sparse_output=False
        )
    )
])
```

## What information is preserved?

The representation preserves information about:

- property size
- number of rooms
- geographical position
- property type
- tenure
- energy rating

The most important loss occurs when raw textual information is converted into structured numerical variables. A full address is not preserved as a raw sentence, but its useful geographical information can still be represented through structured location features.

### **What is the final model input?**

After preprocessing, the categorical variables are expanded into one-hot dimensions and numerical variables are standardized. The result is a dense numerical matrix that can be passed directly to the regression models.

## **5 Exploratory Data Analysis**

Exploratory analysis was used to understand both the target distribution and the relationships between property characteristics and price.

### **House-price distribution**

The historical price distribution is strongly right-skewed. Most observations are concentrated at relatively moderate prices, while a much smaller number of properties have extremely high values.

This matters because the regression problem is not balanced across the target range. A model can perform reasonably well for typical properties while still making very large errors for premium properties.

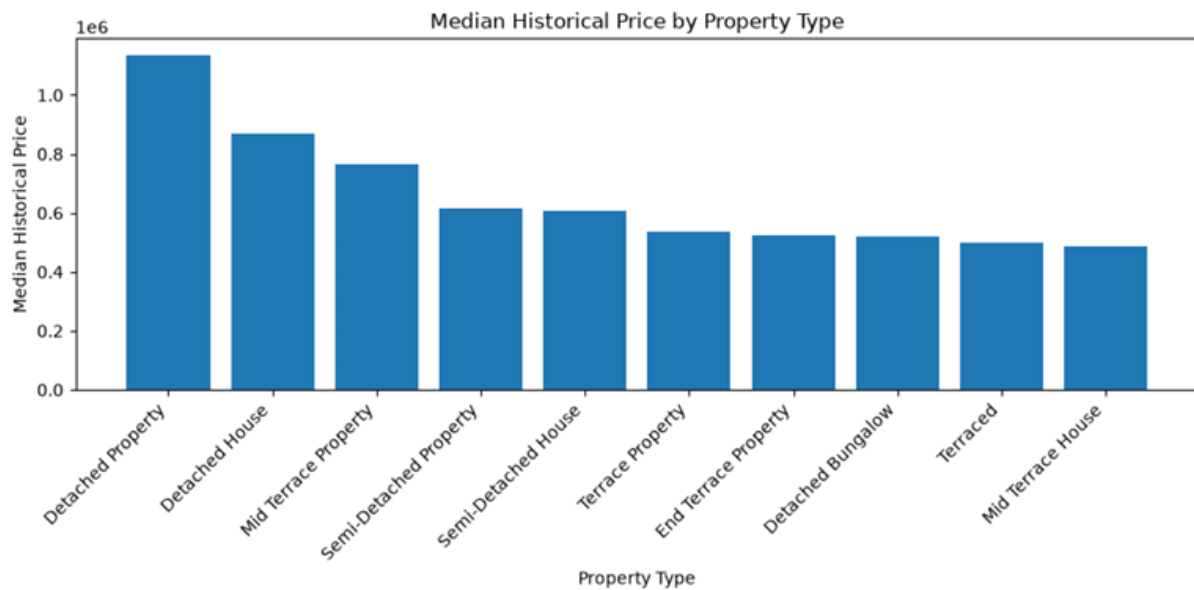
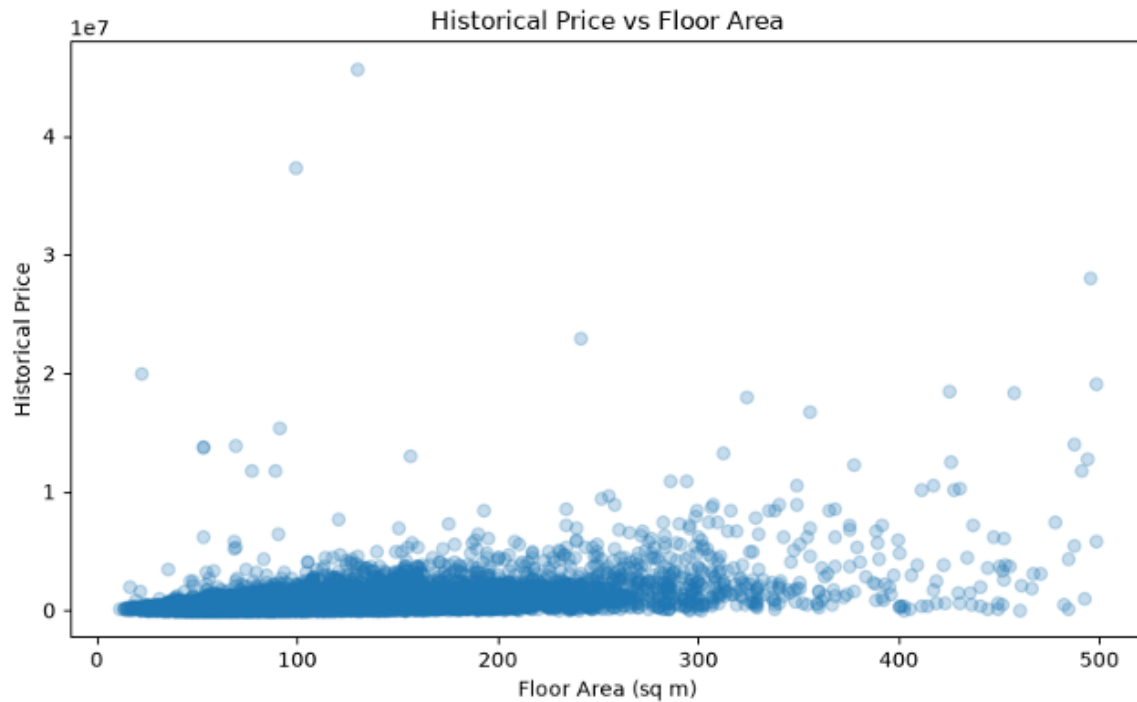


```
count    history_price
count    4.181940e+05
mean     6.395867e+05
std      1.327944e+06
min      1.000000e+04
25%      2.280000e+05
50%      4.000000e+05
75%      6.650000e+05
max      1.058000e+08
```

## Property characteristics and price

Floor area shows a clear positive relationship with historical price. Larger properties generally tend to command higher prices, although the relationship is not perfectly linear.

Room-related variables such as bedrooms, bathrooms and living rooms also show positive relationships with price. Their effect is less direct than floor area because the same number of rooms can occur in properties with very different sizes and locations.

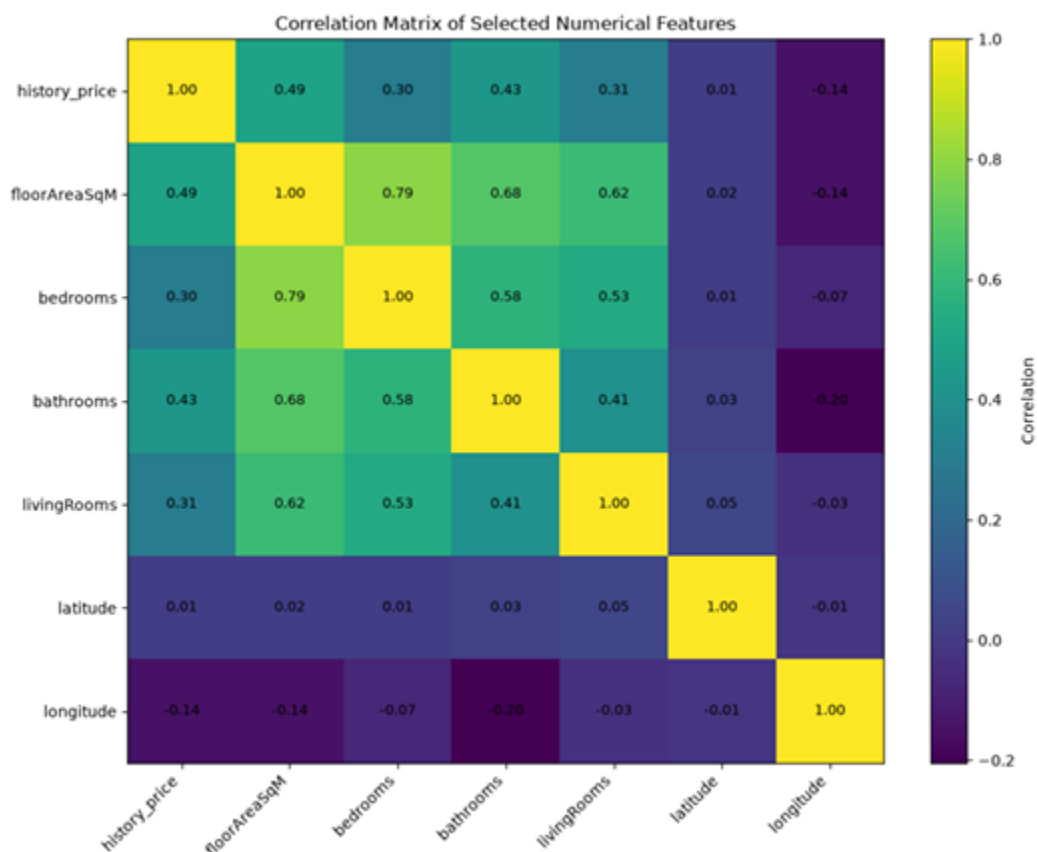


## Correlation analysis

The correlation matrix shows that floor area has the strongest positive linear relationship with historical price among the numerical property variables. Room-

related variables also have positive associations with price, while the geographical coordinates have weaker linear correlations.

Correlation should not be interpreted as a complete description of feature importance. A weak linear correlation does not imply that a variable has no predictive value, particularly when its relationship with the target is nonlinear.



## ML implication

The EDA provides a practical explanation for the model selection. Price is influenced by multiple interacting variables, the target is highly skewed, and some relationships are clearly nonlinear. These characteristics make it reasonable to compare linear regression with nonlinear tree-based ensemble methods.

## 6 Regression Models

Five regression models were developed and compared:

1. Linear Regression

2. Ridge Regression
3. Decision Tree Regressor
4. Random Forest Regressor
5. Gradient Boosting Regressor

Linear Regression provides a simple baseline for modelling the relationship between the features and the target. Ridge Regression extends this approach by introducing regularisation. Decision Tree provides a nonlinear model based on recursive splits, while Random Forest and Gradient Boosting combine multiple trees to obtain more expressive predictive functions.

The models were trained using the same processed feature representation so that the comparison reflects differences in modelling behaviour rather than differences in preprocessing.

```
models = {  
    "Linear Regression":  
        LinearRegression(),
```

```
"Ridge":
    Ridge(
        alpha=1.0
    ),

"Decision Tree":
    DecisionTreeRegressor(
        max_depth=12,
        min_samples_leaf=5,
        random_state=RANDOM_SEED
    ),

"Random Forest":
    RandomForestRegressor(
        n_estimators=200,
        max_depth=16,
        min_samples_leaf=2,
        n_jobs=-1,
        random_state=RANDOM_SEED
    ),

"Gradient Boosting":
    GradientBoostingRegressor(
        n_estimators=200,
        learning_rate=0.05,
        max_depth=4,
        random_state=RANDOM_SEED
    )
}
```

```
for model_name, model in models.items():

    print(
        f"Training {model_name}..."
    )

    start_time = (
        time.perf_counter()
    )

    model.fit(

        X_train_processed,

        y_train

    )

    training_time = (

        time.perf_counter()
        -
        start_time

    )

    prediction = (

        model
```

```

        .predict(
            X_val_processed
        )

    )

    model_results.append({

        "Model":
            model_name,

        "MAE":
            mean_absolute_error(
                y_val,
                prediction
            ),

        "RMSE":
            np.sqrt(
                mean_squared_error(
                    y_val,
                    prediction
                )
            ),

        "R2":
            r2_score(
                y_val,
                prediction
            ),

        "Training Time (s)":
            round(
                training_time,
                4
            )
    })

```

The current validation results are:

|   | Model             | MAE           | RMSE         | R2       | Training Time (s) |
|---|-------------------|---------------|--------------|----------|-------------------|
| 0 | Random Forest     | 152731.198848 | 8.619428e+05 | 0.585533 | 485.7822          |
| 1 | Gradient Boosting | 184571.644547 | 8.876463e+05 | 0.560445 | 749.3185          |
| 2 | Decision Tree     | 197778.677719 | 9.531536e+05 | 0.493174 | 8.5116            |
| 3 | Linear Regression | 294794.548124 | 1.078648e+06 | 0.350928 | 2.0141            |
| 4 | Ridge             | 294747.888309 | 1.078660e+06 | 0.350914 | 0.4980            |

The results show that the nonlinear tree-based models outperform the two linear models. This supports the conclusion from EDA that house prices cannot be adequately described by a simple linear relationship over the available variables.

A notable result is that Random Forest achieves the lowest MAE, while Gradient Boosting achieves the lowest RMSE and the highest  $R^2$ . This difference becomes important in the final model-selection decision.

## **7 Evaluation**

### **Mean Absolute Error**

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

MAE represents the average absolute difference between the actual price and the predicted price. Because it is expressed in pounds, it has a direct practical interpretation. A lower MAE means that the model is closer to the actual price on average.

### **Mean Squared Error**

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

MSE gives greater weight to large errors because the residual is squared.

### **Root Mean Squared Error**

$$RMSE = \sqrt{MSE}$$

RMSE is expressed in pounds, making it easier to interpret than MSE. It is especially useful here because a small number of very large house-price errors can have a significant effect on model quality.

### **$R^2$**

$R^2$  measures the proportion of variance in the target that is explained by the regression model.

The selected model currently achieves:

$$R^2 = 0.6239$$

on the validation set. This indicates that the model explains approximately 62.4% of the observed variation in the validation prices.

### **Which model performed best?**

The current validation results identify Gradient Boosting Tuned as the strongest overall model under the chosen selection criterion. It has the lowest RMSE and the highest  $R^2$  among the five candidates.

Random Forest has a lower MAE, meaning its average absolute error is smaller. However, its higher RMSE indicates that it makes larger individual errors more often or with greater magnitude.

### **Why was Gradient Boosting selected?**

The model was selected using validation RMSE. This decision is intentional rather than arbitrary. The target distribution contains very expensive properties, and large prediction errors on such observations are important. RMSE penalises these large errors more heavily than MAE.

The notebook explicitly follows the principle that model selection should be performed using the validation data while the test set remains isolated until the final evaluation.

### **Error analysis**

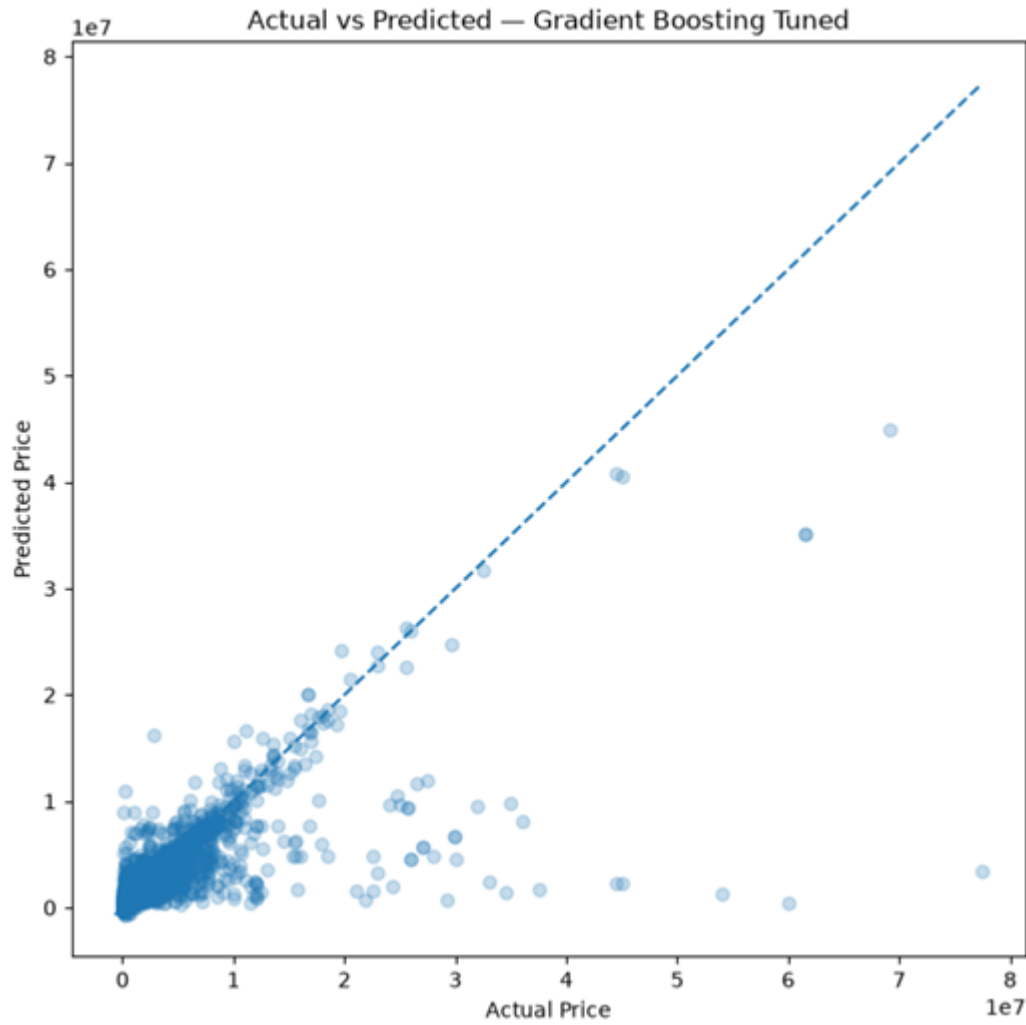
The model performs substantially better for the central portion of the price distribution than for extreme high-value properties. The largest residuals occur when very expensive properties are predicted at much lower prices than their actual values.

This behaviour is visible in the actual-versus-predicted scatter plot. A strong model would place observations close to the diagonal line:

$$\hat{y} = y$$

Large vertical deviations from the diagonal correspond to large prediction errors.

The result suggests that the main limitation is not a complete failure to identify normal property prices, but difficulty modelling the relatively rare upper tail of the market.



```
[38]: # =====
# Largest prediction errors
# =====

error_analysis_df = pd.DataFrame({

    "Actual Price":
        y_val.to_numpy(),

    "Predicted Price":
        selected_validation_prediction,

    "Residual":
        residuals,

    "Absolute Error":
        np.abs(residuals)

})

display(

    error_analysis_df

    .sort_values(
        "Absolute Error",
        ascending=False
    )
)
```

```
)

.head(10)

)
```

|       | Actual Price | Predicted Price | Residual     | Absolute Error |
|-------|--------------|-----------------|--------------|----------------|
| 27472 | 77500000     | 3.404110e+06    | 7.409589e+07 | 7.409589e+07   |
| 60072 | 60000000     | 3.722133e+05    | 5.962779e+07 | 5.962779e+07   |
| 40808 | 54000000     | 1.351699e+06    | 5.264830e+07 | 5.264830e+07   |
| 13809 | 45000000     | 2.287829e+06    | 4.271217e+07 | 4.271217e+07   |
| 46896 | 44500000     | 2.262281e+06    | 4.223772e+07 | 4.223772e+07   |
| 7553  | 37500000     | 1.641827e+06    | 3.585817e+07 | 3.585817e+07   |
| 52996 | 34500000     | 1.363616e+06    | 3.313638e+07 | 3.313638e+07   |
| 26749 | 33000000     | 2.364182e+06    | 3.063582e+07 | 3.063582e+07   |
| 17588 | 29160447     | 6.504100e+05    | 2.851004e+07 | 2.851004e+07   |
| 22659 | 36000000     | 8.162777e+06    | 2.783722e+07 | 2.783722e+07   |

## 8 Deployment

The final model is deployed as both a web application and a mobile application

The deployment architecture follows:

User Input → Validation → Preprocessing → Saved Model → Prediction

The same preprocessing logic used during training is preserved with the model so that raw user input is converted into exactly the representation expected by the regression model.

### **Web deployment**

The Web application is implemented with Flask. The service loads the saved model pipeline and performs inference on new property information rather than retraining the model.

The input interface provides the property and location information required by the deployed representation. After validation, the input is passed through the saved preprocessing pipeline and then to the selected regression model.

The main output is:

Predicted House Price

The interface can also display supporting model information such as the selected model and validation performance, but these values describe model performance and should not be presented as a probability that the individual price estimate is

correct.

The screenshot shows the 'London House Price' mobile app interface. The title bar is dark blue with the text 'London House Price' and 'Machine Learning Property Estimator'. Below the title bar is a light blue header with the text 'Property Details' and two buttons: 'Load Sample' and 'Clear'. The main form area is white and contains several input fields: 'Location' (Latitude: 51.501521, Longitude: -0.108597, Outcode: E1), 'Property Characteristics' (Floor Area: 84.0 m², Bedrooms: 2.0, Bathrooms: 1.0, Living Rooms: 1.0, Property Type: Bungalow Property, Tenure: Feudal, Energy Rating: A). The form is organized into sections with labels like 'Location', 'Property Characteristics', 'Living Rooms', and 'Energy Rating'.

The screenshot shows the 'London House Price' mobile app interface displaying the estimated historical price. The title bar is dark blue with the text 'London House Price' and 'Machine Learning Property Estimator'. Below the title bar is a light blue header with the text 'Estimated Historical Price' and a button: 'Estimate House Price'. The main content area is white and features a large, bold, dark blue text displaying the estimated historical price: '£1,180,468.08'. Below the price, there are four boxes showing validation metrics: 'Model Confidence Indicator' (62.39%, Based on validation R², not a probability), 'Validation R²' (0.624, Explained variance), 'Validation MAE' (£170,394.50, Typical absolute error), and 'Validation RMSE' (£821,122.40, Sensitive to large errors). At the bottom, there is a section for the 'Model' (Gradient Boosting Tuned) and a disclaimer: 'This estimate is produced by a machine-learning model and should not be treated as a professional property valuation.' The footer of the app is dark blue with the text 'London House Price Prediction System'.

## Mobile deployment

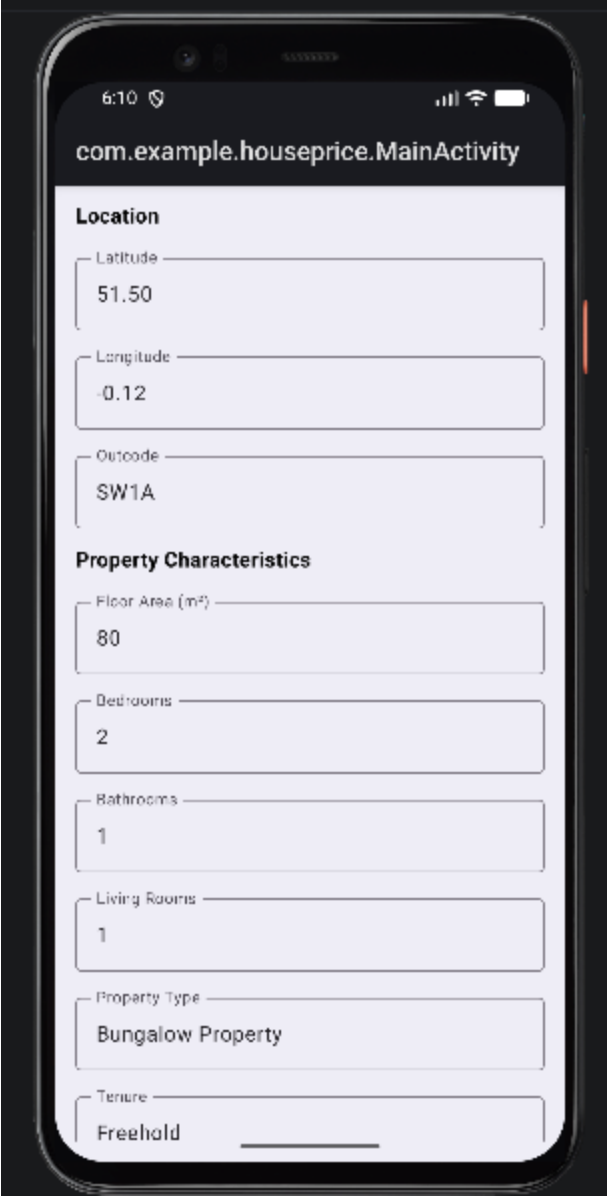
The mobile application is implemented using Android and Kotlin. The mobile client does not train or store the regression model locally. Instead, it communicates with the Flask REST service.

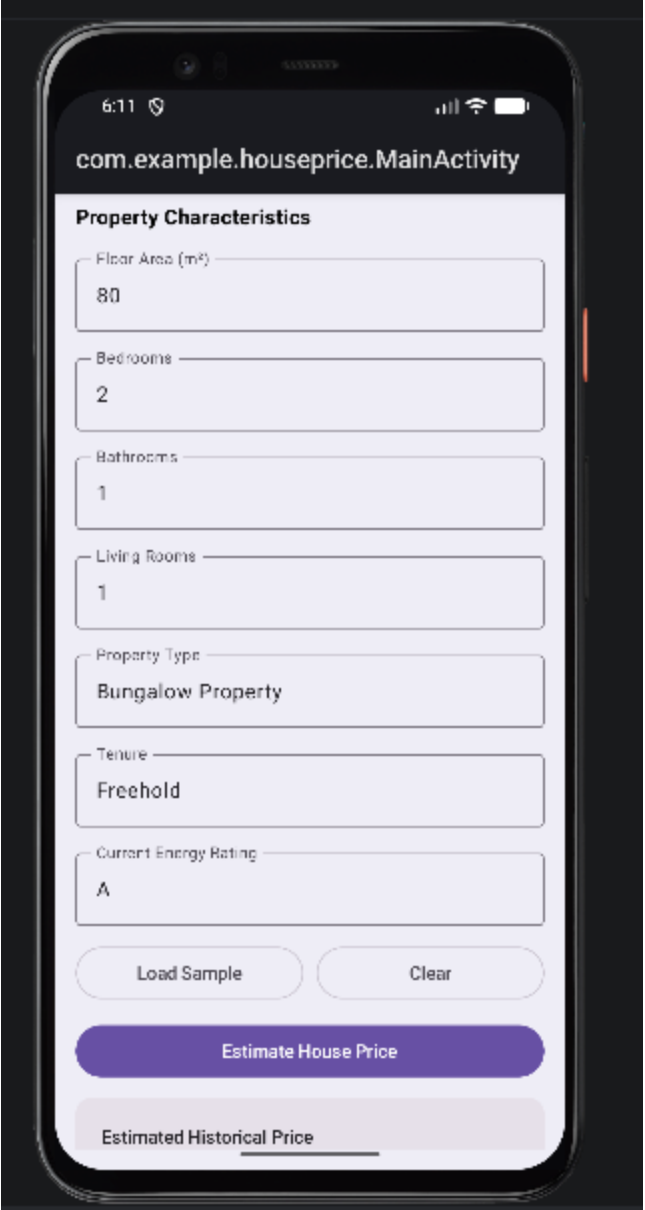
The complete workflow is:

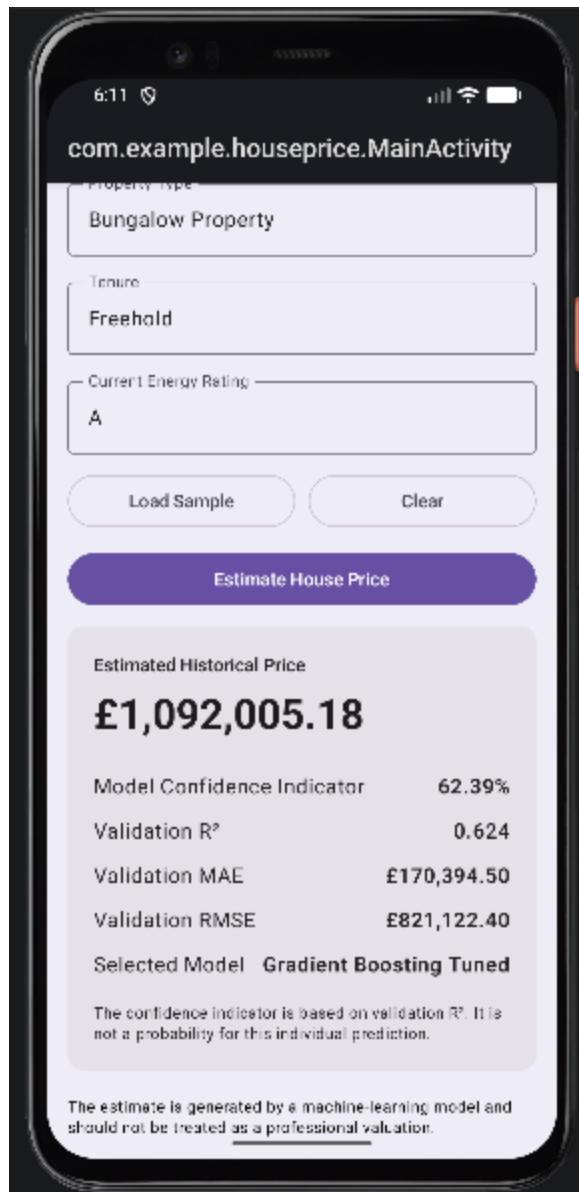
Mobile UI → REST API → Preprocessing → Saved ML Model → Prediction  
→ Mobile UI

This follows the recommended mobile architecture in Assignment 02.

The mobile application allows the user to enter or select the required property characteristics, submit a prediction request and display the resulting estimated house price.







## Deployment considerations

A key requirement is consistency between training and inference. If a categorical value is encoded differently, a numerical variable is scaled differently, or the feature order changes, the deployed model will receive a different representation from the one used during training. The prediction can then become unreliable even though the model itself has not changed.

## C. Application 3 — E-Commerce Customer Behavior and Interest

### 1 Problem Description

The objective of this application is to analyze historical customer behavior and discover the product category that a customer is most likely to purchase in a future transaction.

The application is formulated as a supervised multi-class classification problem. Instead of treating every transaction as an independent sample, the system constructs a customer-level observation from the customer's historical purchasing activity. The model then predicts the product category associated with the customer's next observed purchase.

The learning task is defined as:

$X$  = customer behavioral features + review text representation

$y$  = product category of the target purchase

The target is therefore a categorical variable. Each customer snapshot belongs to one product-category class.

The system also contains a secondary customer-behavior prediction component that classifies the expected target-order value into Low, Medium, or High. This secondary output is used as an additional customer signal, while product-category prediction remains the primary interest-discovery task.

The complete application follows the sequence:

Customer History → Customer Profile → Numerical Representation  
→ Machine Learning Model → Predicted Interest  
→ Product Recommendation

### Definition of one observation

One observation represents one customer who has at least two delivered orders.

The notebook follows a temporal prediction design:

Previous orders →  $X$

Final delivered order  $\rightarrow y$

For every eligible customer, the chronologically last delivered order is treated as the target order. All earlier delivered orders form the historical information used to construct the customer profile. The notebook identifies 2,801 eligible customers and 5,921 eligible delivered orders.

This formulation is preferable to using the complete customer history as both input and target because it preserves the direction of prediction. The model is trained to infer future interest from information that existed before the target event.

### **Definition of $X$ and $y$**

The input is:

$$X = [X_{tabular}, X_{text}]$$

where  $X_{tabular}$  contains structured behavioral and customer attributes, while  $X_{text}$  contains the numerical representation of historical customer review comments.

The target is:

$$y = \text{target product category}$$

### **Why this problem is useful**

The classification output can support several practical e-commerce decisions.

A predicted customer interest can be used to select relevant products, prioritize personalized content, support targeted promotions, and improve customer segmentation. The predicted behavior level provides an additional indication of the customer's potential economic value.

The final application therefore moves beyond descriptive analytics. It transforms historical transactions into an actionable customer-level prediction.

## **2 Dataset**

**Dataset name:** Brazilian E-Commerce Public Dataset by Olist

**Source:** Kaggle

**Kaggle URL:**

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

The dataset is relational rather than a single flat table. The notebook loads the following CSV datasets:

| Table       | Rows      | Columns | Role                                   |
|-------------|-----------|---------|----------------------------------------|
| Reviews     | 99,224    | 7       | Customer review and score information  |
| Orders      | 99,441    | 8       | Transaction and order timestamps       |
| Products    | 32,951    | 9       | Product attributes and categories      |
| Sellers     | 3,095     | 4       | Seller information                     |
| Translation | 71        | 2       | Portuguese to English category mapping |
| Customers   | 99,441    | 5       | Customer identity and state            |
| Geolocation | 1,000,163 | 5       | Geographic reference data              |
| Order Items | 112,650   | 7       | Product-level order details            |
| Payments    | 103,886   | 5       | Payment information                    |

The main customer-interest pipeline uses Customers, Orders, Order Items, Products, Payments, Reviews, and Product Category Translation. Sellers and Geolocation are inspected but are not required for the main prediction pipeline.

Loaded tables:

|             |               |
|-------------|---------------|
| reviews     | -> (99224, 7) |
| orders      | -> (99441, 8) |
| products    | -> (32951, 9) |
| sellers     | -> (3095, 4)  |
| translation | -> (71, 2)    |



|             |                 |
|-------------|-----------------|
| customers   | -> (99441, 5)   |
| geolocation | -> (1000163, 5) |
| order_items | -> (112650, 7)  |
| payments    | -> (103886, 5)  |

### Why multiple tables are required

Customer behavior is inherently relational.

An order identifies when a customer purchased. Order items identify what was purchased. Products identify the product category. Payments describe transaction value and payment behavior. Reviews provide qualitative feedback.

The relationships are therefore represented conceptually as:

*customer → orders → order items → products*

and:

*orders → payments*

*orders → reviews*

These multiple data sources are aggregated into a single customer-level modeling representation.

### **Dataset size used for supervised learning**

The raw dataset contains tens of thousands of transactions and related records. After temporal filtering and customer-history construction, the notebook produces:

| <b>Property</b>                              | <b>Value</b> |
|----------------------------------------------|--------------|
| Delivered orders                             | 96,478       |
| Customers with at least two delivered orders | 2,801        |
| Eligible delivered orders                    | 5,921        |
| Customer snapshots                           | 2,801        |
| Raw customer-level columns                   | 52           |
| Raw model input columns                      | 45           |

The distinction between 52 customer-level columns and 45 model input columns is important. The customer-level table contains identifiers and target-related fields that are deliberately excluded from model training.

### **Target definition**

The primary target is the product category of the customer's final delivered order.

The target contains 11 classes after the training-based category mapping:

- beleza\_saude
- brinquedos
- cama\_mesa\_banho
- esporte\_lazer
- fashion\_bolsas\_e\_acessorios
- informatica\_acessorios
- moveis\_decoracao

- other
- relógios\_presentes
- telefonia
- utilidades\_domesticas

Rare categories outside the selected training categories are mapped to the other class for model training.

This produces an imbalanced multi-class classification problem. In the training set, the largest class is other with 622 observations, followed by cama\_mesa\_banho with 274 observations and esporte\_lazer with 182 observations. The smallest modeled training classes include telefonia with 59 observations and brinquedos with 62 observations.

### 3 Customer Representation

#### From transactions to customer profile

The central representation process is:

Transactions → Customer History → Customer Profile → Feature Vector

This is necessary because machine-learning classifiers require a fixed computational representation, while customer behavior naturally appears as a sequence of transactions distributed across several relational tables.

One customer can have many historical orders. The notebook converts this variable-length history into a fixed set of numerical statistics.

The customer-level dataset contains 52 columns before removing identifiers and target variables. After removing non-predictive fields, the model receives 45 raw input columns:

$$X_{raw} \in \mathbb{R}^{N \times 45}$$

where:

$$N = 2801$$

represents the number of supervised customer snapshots and 45 represents the number of raw model-input fields before one-hot encoding and text transformation.

### **Behavioral representation**

A simplified mathematical description is:

$$x_i = [R_i, F_i, M_i, C_{i1}, C_{i2}, \dots, C_{ik}, P_i, V_i, S_i]$$

where:

- $R_i$  represents recency
- $F_i$  represents purchase frequency
- $M_i$  represents monetary behavior
- $C_{ij}$  represents customer activity in category  $j$
- $P_i$  represents payment behavior
- $V_i$  represents review behavior
- $S_i$  represents customer profile information

The actual implementation expands this concept into 42 numerical features, 2 categorical features, and 1 text feature.

### **Numerical features**

The 42 numerical features are:

- prior\_order\_count
- total\_spending
- average\_order\_value
- max\_order\_value
- recency\_days
- history\_duration\_days
- average\_days\_between\_orders
- min\_days\_between\_orders

- max\_days\_between\_orders
- total\_items
- average\_item\_price
- average\_freight\_value
- average\_items\_per\_order
- category purchase counts for selected categories
- category purchase shares for selected categories
- average\_installments
- average\_payment\_value
- average\_review\_score
- review\_count
- text\_review\_count

The category representation contains both absolute activity and relative activity.

A category count records how many historical items a customer purchased in a category.

A category share measures the proportion of modeled category activity associated with that category.

For category  $j$ :

$$C_{ij} = \text{number of historical items purchased in category } j$$

and:

$$S_{ij} = \frac{C_{ij}}{\sum_k C_{ik}}$$

This distinction is useful because raw purchase counts alone do not distinguish customer scale from customer preference.

```
numeric_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="median"
        )
    ),
    (
        "scaler",
        StandardScaler()
    )
])
```

## Temporal features

Recency is calculated as:

$$R_i = t_{target,i} - t_{last\ history,i}$$

measured in days.

A smaller recency value indicates that the customer's previous purchase was close to the target purchase in time.

The notebook also computes:

$$history\ duration = t_{last\ history} - t_{first\ history}$$

and statistics describing the time gaps between historical orders.

These variables allow the model to distinguish customers who buy frequently from customers whose purchases are widely separated in time.

## Categorical features

The two categorical input variables are:

- customer\_state
- dominant\_payment\_type

These values are not inserted directly into the classifier as strings. They are transformed using one-hot encoding.

Conceptually, if a categorical field contains:

{credit card, boleto, voucher}

then one observation is converted to a binary representation such as:

[1, 0, 0]

The order of the categories has no mathematical meaning. The purpose is to indicate membership.

The encoder uses unknown-category protection, allowing unseen categories during deployment to be ignored rather than causing the service to fail.

```
categorical_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="most_frequent"
        )
    ),
    (
        "onehot",
        create_dense_one_hot_encoder()
    )
])
```

## Text representation

Historical customer review comments form the second information modality.

The raw review comment is cleaned before being used. The cleaning process:

- converts text to lowercase;
- normalizes whitespace;
- removes punctuation;
- preserves the remaining word content.

Reviews belonging to the same customer are concatenated to form a customer-level review\_text field.

The representation is:

review comments → cleaned text → TF-IDF vectors

The final combined pipeline uses two complementary TF-IDF representations:

word TF-IDF + character TF-IDF

Word-level TF-IDF uses:

- maximum vocabulary of 3,000 features
- minimum document frequency of 1
- maximum document frequency of 0.98
- unigram and bigram features
- sublinear term frequency

Character-level TF-IDF uses:

- character-within-word analysis
- maximum vocabulary of 3,000 features
- minimum document frequency of 1
- maximum document frequency of 0.98
- character n-grams from length 3 through length 5
- sublinear term frequency

The combined representation therefore captures both recognizable words and smaller character patterns.

```
word_tfidf = TfidfVectorizer(  
    analyzer="word",  
    max_features=3000,  
    min_df=1,  
    max_df=0.98,  
    ngram_range=(1, 2),  
    sublinear_tf=True  
)
```

```
# -----  
# Character-level TF-IDF  
# -----
```

```
char_tfidf = TfidfVectorizer(  
    analyzer="char_wb",  
    max_features=3000,  
    min_df=1,  
    max_df=0.98,  
    ngram_range=(3, 5),  
    sublinear_tf=True  
)
```

```
word_pipeline = Pipeline([  
    (  
        "flatten",  
        FunctionTransformer(  
            text_to_1d,  
            validate=False  
        )  
    ),  
  
    (  
        "tfidf",  
        word_tfidf  
    )  
)
```

```

])

char_pipeline = Pipeline([
    (
        "flatten",
        FunctionTransformer(
            text_to_id,
            validate=False
        )
    ),

    (
        "tfidf",
        char_tfidf
    )
])

```

## Final computational representation

The final customer representation is:

$$X = [X_{tabular}; X_{word}; X_{char}]$$

The notebook transforms five sample records into:

(5, 6073)

which means that each transformed customer observation contains 6,073 numerical dimensions before entering the final classifier.

The data type is a sparse CSR matrix.

The dimensions have the following meaning:

- 5 represents the number of sample customers transformed in the inspection step
- 6,073 represents the final number of computational features produced by the complete preprocessing pipeline

The 6,073 dimensions are larger than the original 45 raw columns because categorical variables are expanded by one-hot encoding and review text is converted into high-dimensional TF-IDF features.

This illustrates an essential distinction between raw data and model input:

|                                                                        |
|------------------------------------------------------------------------|
| <i>45 raw fields <math>\neq</math> 6073 transformed model features</i> |
|------------------------------------------------------------------------|

The raw customer table therefore describes the customer at a human-readable level, while the transformed feature matrix represents the same customer in a numerical space that the machine-learning model can process.

```
Number of raw input columns: 45
```

```
X_train shape: (1960, 45)
```

```
X_val shape: (420, 45)
```

```
X_test shape: (421, 45)
```

```
X_train_text shape: (1960, 1)
```

```
X_val_text shape: (420, 1)
```

```
X_test_text shape: (421, 1)
```

```
Target classes:
```

1. beleza\_saude
2. brinquedos
3. cama\_mesa\_banho
4. esporte\_lazer
5. fashion\_bolsas\_e\_acessorios
6. informatica\_acessorios
7. moveis\_decoracao
8. other
9. relorios\_presentes
10. telefonia
11. utilidades\_domesticas

Training target distribution:

|                             | count |
|-----------------------------|-------|
| target_category             |       |
| other                       | 622   |
| cama_mesa_banho             | 274   |
| esporte_lazer               | 182   |
| moveis_decoracao            | 180   |
| beleza_saude                | 164   |
| informatica_acessorios      | 139   |
| utilidades_domesticas       | 115   |
| relogios_presentes          | 94    |
| fashion_bolsas_e_acessorios | 69    |
| brinquedos                  | 62    |
| telefonica                  | 59    |

Review text statistics:

Customers with non-empty review text: 547

Customers with empty review text: 1413

Average review text length: 17.62

## 4 Data Cleaning

### Missing values

The notebook investigates missing values before cleaning.

Missing data are handled according to the semantics of the variable.

For review text, missing comments are converted to an empty string because the absence of a comment is meaningful and can naturally be represented as no textual content.

For customer state, missing values are mapped to unknown.

Numerical missing values are not manually filled at the data-table stage. Instead, they are handled consistently inside the preprocessing pipeline using median imputation.

This avoids mixing exploratory data preparation with model-specific preprocessing.

```
=====
CUSTOMERS
```

```
=====
No missing values.
```

```
=====
ORDERS
```

```
=====
                                     missing_count  missing_percent
order_delivered_customer_date          2965           2.981668
order_delivered_carrier_date            1783           1.793023
order_approved_at                       160            0.160899
```

```
=====
ORDER_ITEMS
```

```
=====
No missing values.
```

```
=====
PRODUCTS
```

```
=====
                                     missing_count  missing_percent
product_category_name          610           1.851234
product_description_lenght      610           1.851234
product_name_lenght             610           1.851234
product_photos_qty              610           1.851234
product_weight_g                 2            0.006070
product_height_cm                2            0.006070
```

```
=====
PAYMENTS
```

```
=====
No missing values.
```

```
=====
REVIEWS
```

```
=====
                                     missing_count  missing_percent
review_comment_title            87656           88.341530
review_comment_message          58247           58.702532
```

## Duplicate records

The notebook investigates both exact duplicates and business-key duplicates.

The main uniqueness constraints are:

- customers by customer\_id
- orders by order\_id
- order items by order\_id and order\_item\_id
- products by product\_id
- payments by order\_id and payment\_sequential
- reviews by review\_id

Cleaning is therefore based on the semantics of the business entity rather than simply deleting every repeated-looking row.

|   | table     | exact_duplicate_rows | business_key_duplicates |
|---|-----------|----------------------|-------------------------|
| 0 | customers | 0                    | 0                       |

29

|   |             |   |     |
|---|-------------|---|-----|
| 1 | orders      | 0 | 0   |
| 2 | order_items | 0 | 0   |
| 3 | products    | 0 | 0   |
| 4 | payments    | 0 | 0   |
| 5 | reviews     | 0 | 814 |

## Invalid transactions

Only delivered orders are retained for the main behavioral-history construction.

For order items, the following validity rules are applied:

$$\begin{aligned} price &> 0 \\ freight\_value &\geq 0 \end{aligned}$$

For payments:

$$payment\_value > 0$$

These constraints prevent zero or physically implausible monetary observations from distorting behavioral features.

**Cleaned table sizes:**

|                                |                                |
|--------------------------------|--------------------------------|
| <code>orders_clean</code>      | <code>-&gt; (96478, 8)</code>  |
| <code>order_items_clean</code> | <code>-&gt; (112650, 7)</code> |
| <code>products_clean</code>    | <code>-&gt; (32951, 9)</code>  |
| <code>payments_clean</code>    | <code>-&gt; (103877, 5)</code> |
| <code>reviews_clean</code>     | <code>-&gt; (98410, 7)</code>  |

## **Inconsistent category representation**

Product categories are stored in Portuguese in the original dataset. The notebook uses a translation table to provide English category names for exploratory analysis and recommendation catalog presentation.

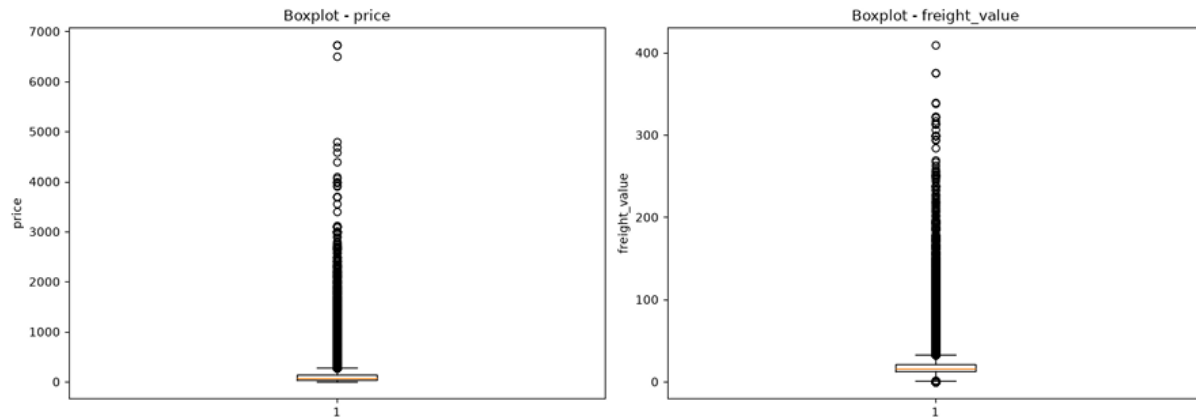
The underlying target representation remains based on the original category labels used by the modeling pipeline.

## **Extreme purchasing behavior**

The notebook performs price and freight-value outlier analysis using boxplots and quantiles.

For visualization of product price, values up to the 99th percentile are used so that a small number of extreme purchases do not visually dominate the distribution.

Extreme behavior is therefore examined rather than blindly removed. The intention is to distinguish genuine high-value purchasing behavior from invalid records.



## Temporal cleaning

The most important cleaning operation for this application is temporal filtering.

Historical order information is restricted to:

$$t_{history} < t_{target}$$

Review information is restricted to reviews available on or before the target timestamp when the review creation timestamp is available.

This is critical because future information would create data leakage.

## 5 Interest Discovery

### Customer interest as a behavioral pattern

Customer interest is not represented by one isolated variable.

Instead, the model observes a combination of:

frequency + recency + monetary behavior + category activity + review behavior  
+ text

This allows the system to capture several dimensions of customer preference simultaneously.

### Purchase frequency

prior\_order\_count measures how many previous delivered orders are available for the customer.

A high value indicates repeated interaction with the platform.

Repeated purchases provide the model with more behavioral evidence than a customer with only one historical order.

### **Monetary behavior**

The main monetary features are:

- total\_spending
- average\_order\_value
- max\_order\_value
- average\_payment\_value

These distinguish customers who spend frequently, customers who make expensive purchases, and customers with relatively low transaction values.

### **Recency**

Recency indicates how recently the customer purchased before the target order.

A customer with a very recent historical purchase may exhibit a different purchasing pattern from a customer whose previous purchase occurred much earlier.

The feature is therefore useful for identifying active versus less active customers.

### **Category activity**

Category counts and category shares provide the most direct behavioral description of product preference.

For example, a customer with repeated historical activity in:

*informatica\_acessorios*

has explicit historical evidence of interest in that category.

However, interest is not deterministic. A customer may purchase one category repeatedly and then switch to another category in the next order. The classification model is therefore estimating the most plausible target category rather than reproducing historical behavior exactly.

### **Review behavior and text**

Review scores provide numerical feedback behavior, while review text provides qualitative information.

A review such as a positive statement about quality contains semantic information that cannot be fully represented by purchase count or spending alone.

In the current model, this information is encoded numerically through TF-IDF rather than through a semantic transformer.



### **Observed target distribution**

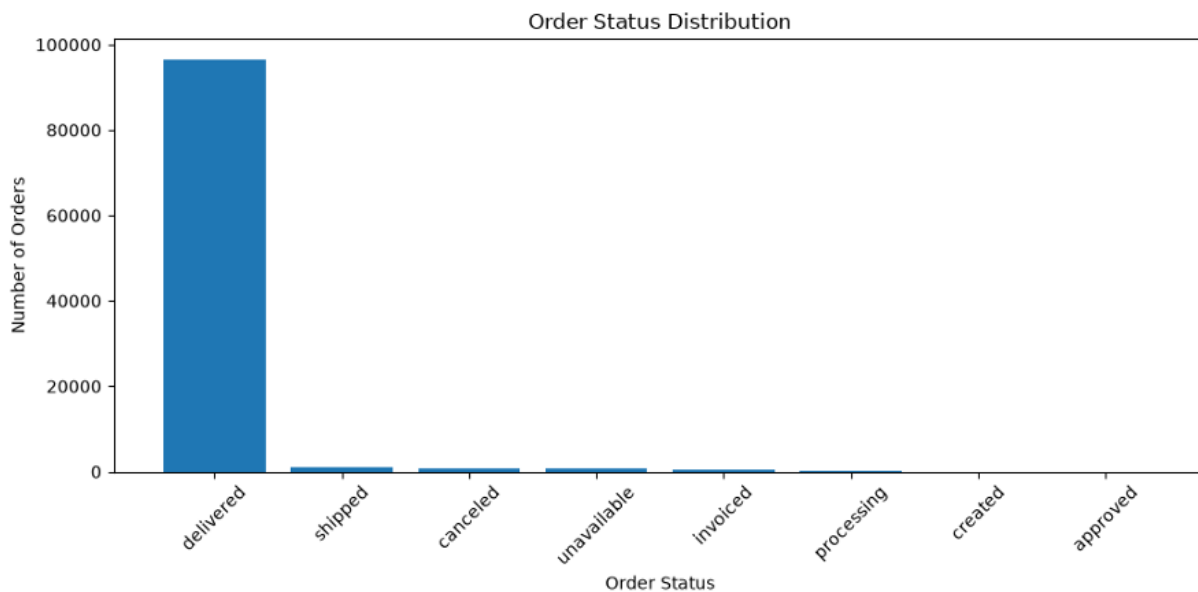
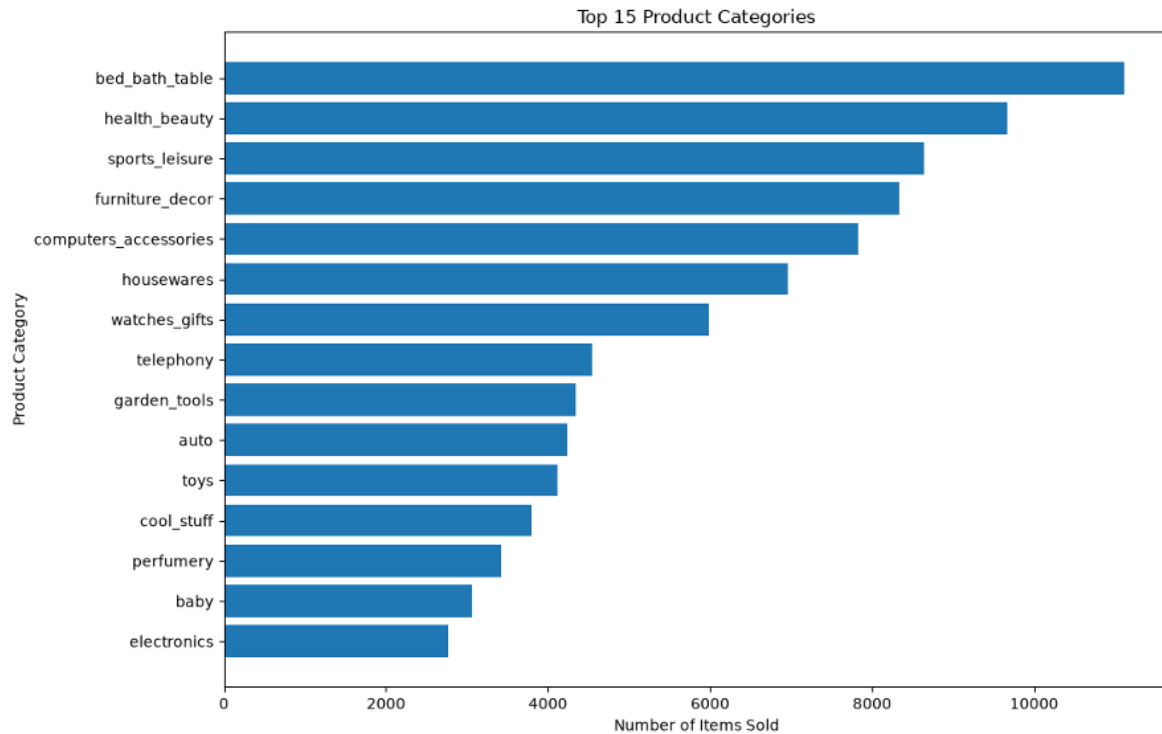
The stratification stage produces 11 modeled target classes.

The largest class is other with 891 observations in the split-control distribution.

Among explicit categories, cama\_mesa\_banho has 392 observations, esporte\_lazer has 260, moveis\_decoracao has 257, and beleza\_saude has 235.

Smaller categories include automotivo with 81 and brinquedos with 89.

The imbalance explains why Macro F1 is used as the primary model-selection metric rather than relying on accuracy alone.



## 10.6 Model Development

### Baseline

Before training machine-learning models, the notebook establishes a `DummyClassifier` baseline using the most-frequent strategy.

```

baseline_model = DummyClassifier(
    strategy="most_frequent"
)

print("Training DummyClassifier baseline...")

baseline_model.fit(
    X_train,
    y_train
)

baseline_prediction = (
    baseline_model.predict(
        X_val
    )
)

```

The baseline results are:

```
Training DummyClassifier baseline...
```

|   | Model           | Strategy      | Accuracy | Precision Macro | Recall Macro | \ |
|---|-----------------|---------------|----------|-----------------|--------------|---|
| 0 | DummyClassifier | most_frequent | 0.319048 | 0.029004        | 0.090909     |   |

```

F1 Macro
0 0.043978

```

The baseline always predicts the most frequent training category.

Its Macro F1 of only 0.044 demonstrates that simply favoring the largest class performs poorly when equal importance is given to all categories.

## Six-model comparison

The first model-development stage evaluates six required model families:

1. Logistic Regression
2. Decision Tree
3. Random Forest
4. SVM

## 5. Text-based Linear Classifier

## 6. Gradient Boosting

The six models provide a baseline comparison across structured and text-only representations.

The validation results are:

| <b>Model</b>                    | <b>Accuracy</b> | <b>Precision<br/>Macro</b> | <b>Recall<br/>Macro</b> | <b>Macro<br/>F1</b> | <b>Weighted<br/>F1</b> | <b>ROC-<br/>AUC</b> |
|---------------------------------|-----------------|----------------------------|-------------------------|---------------------|------------------------|---------------------|
| Random Forest                   | 0.4976          | 0.4859                     | 0.4619                  | <b>0.4705</b>       | 0.4942                 | 0.7489              |
| SVM                             | 0.4881          | 0.4709                     | 0.4495                  | 0.4549              | 0.4880                 | 0.7144              |
| Logistic<br>Regression          | 0.4429          | 0.4111                     | 0.4322                  | 0.4138              | 0.4489                 | 0.7344              |
| Decision Tree                   | 0.4643          | 0.4416                     | 0.4038                  | 0.4080              | 0.4653                 | 0.7098              |
| Gradient<br>Boosting            | 0.4548          | 0.4363                     | 0.3509                  | 0.3794              | 0.4383                 | 0.7427              |
| Text-based<br>Linear Classifier | 0.1119          | 0.1204                     | 0.1128                  | 0.0649              | 0.0723                 | 0.5212              |

The Random Forest provides the strongest baseline performance.

Compared with the DummyClassifier baseline:

$$0.4705 - 0.0440 = 0.4265$$

Macro F1 therefore improves by 0.4265 absolute points.

This confirms that the behavioral feature representation contains substantial predictive information even before review text is incorporated.

### **Combined Tabular + Review Text models**

The second stage evaluates classifiers using the combined representation:

Tabular Features + Word TF-IDF + Character TF-IDF

Four classifiers are compared:

- Logistic Regression + Text
- Decision Tree + Text
- Random Forest + Text
- SVM + Text

The validation results are:

| <b>Model</b>                     | <b>Representation</b>    | <b>Accuracy</b> | <b>Precision<br/>Macro</b> | <b>Recall<br/>Macro</b> | <b>Macro F1</b> | <b>Weighted<br/>F1</b> | <b>ROC-<br/>AUC</b> |
|----------------------------------|--------------------------|-----------------|----------------------------|-------------------------|-----------------|------------------------|---------------------|
| SVM +<br>Text                    | Tabular +<br>Review Text | <b>0.4881</b>   | <b>0.4697</b>              | 0.4495                  | <b>0.4544</b>   | <b>0.4881</b>          | 0.7154              |
| Random<br>Forest +<br>Text       | Tabular +<br>Review Text | 0.4476          | 0.4338                     | <b>0.4597</b>           | 0.4364          | 0.4542                 | <b>0.7347</b>       |
| Logistic<br>Regression<br>+ Text | Tabular +<br>Review Text | 0.4500          | 0.4220                     | 0.4479                  | 0.4277          | 0.4549                 | 0.7344              |
| Decision<br>Tree +<br>Text       | Tabular +<br>Review Text | 0.4643          | 0.4388                     | 0.4038                  | 0.4061          | 0.4651                 | 0.7095              |

Among combined models, SVM + Text achieves the highest Macro F1 of:

**0.4544**

and therefore becomes the selected final classifier within the combined representation.

### **Selected final model**

The final model is:

**SVM + Tabular + Review Text**

The SVM uses:

- radial-basis-function kernel
- $C = 2.0$
- balanced class weights
- probability estimation enabled
- random seed 42

"SVM + Text":

```
SVC(  
    kernel="rbf",  
    C=2.0,  
    class_weight="balanced",  
    probability=True,  
    random_state=RANDOM_SEED  
)
```

The probability setting allows the deployed system to return class probabilities rather than only a hard class label.

It is important to distinguish two selection results.

The best Tabular Only model is Random Forest with Macro F1:

0.4705

The best Tabular + Review Text model is SVM + Text with Macro F1:

0.4544

Therefore, in the current experiment, adding review text **does not improve the best overall validation Macro F1 compared with the best Tabular Only model.**

However, within the combined representation, SVM + Text is the strongest candidate and is selected as the final model for the combined customer-interest system.

## 10.7 Evaluation

### 10.7.1 Evaluation metrics

The main evaluation metrics are Accuracy, Precision, Recall, F1-score, and multiclass ROC-AUC.

#### Accuracy

$$Accuracy = \frac{\text{correct predictions}}{\text{all predictions}}$$

Accuracy measures the overall fraction of correct customer-category predictions.

It is useful for understanding global correctness, but it may be overly influenced by the large other class.

#### Precision

$$Precision = \frac{TP}{TP + FP}$$

Precision measures the reliability of a predicted category.

In this application, precision is important because predicting an incorrect customer interest can lead to irrelevant product recommendations.

#### Recall

$$Recall = \frac{TP}{TP + FN}$$

Recall measures how many customers belonging to a category are successfully identified.

High recall is useful when the objective is to discover as many relevant customer interests as possible.

#### F1-score

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

F1 balances precision and recall.

## Macro F1

For  $K$  categories:

$$F1_{macro} = \frac{1}{K} \sum_{k=1}^K F1_k$$

Macro F1 gives every category equal weight.

This is the primary selection metric because the target classes are imbalanced.

## ROC-AUC

The notebook uses one-vs-rest Macro ROC-AUC for the multi-class problem.

This evaluates how effectively the model separates each category from the remaining categories across probability thresholds.

## Final test performance

After model selection, the selected combined model is refitted using the training and validation sets.

The refit dataset contains:

$$1960 + 420 = 2380$$

customer observations.

The model is then evaluated on the untouched test set of 421 customers.

The final test results are:

| Metric          | Test Result   |
|-----------------|---------------|
| Accuracy        | <b>0.4632</b> |
| Precision Macro | <b>0.4144</b> |
| Recall Macro    | <b>0.3851</b> |
| Macro F1        | <b>0.3943</b> |
| Weighted F1     | <b>0.4584</b> |

| Metric | Test Result |
|--------|-------------|
|--------|-------------|

|                   |               |
|-------------------|---------------|
| ROC-AUC OVR Macro | <b>0.7018</b> |
|-------------------|---------------|

The final test Macro F1 of 0.3943 is lower than the validation Macro F1 of 0.4544.

### **Confusion matrix interpretation**

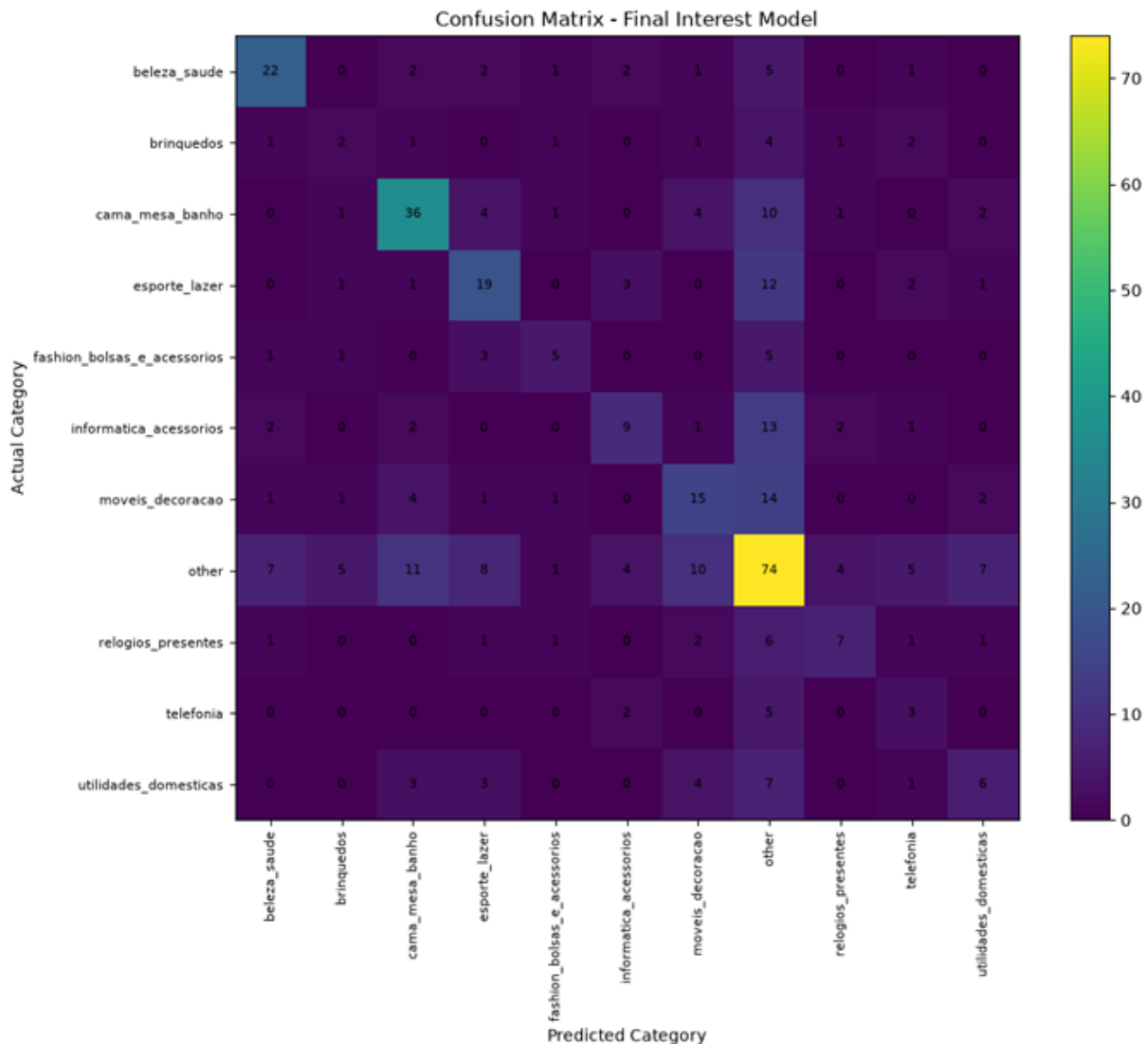
The confusion matrix is used to identify category-level errors.

A correct prediction appears on the diagonal.

An off-diagonal value indicates a customer whose actual target category was assigned to another category.

This analysis is especially important for categories that share purchasing patterns. For example, customers purchasing several types of household-related products may have behavioral histories that make neighboring categories difficult to separate.

The confusion matrix should therefore be interpreted as evidence about the boundaries of the learned representation, not simply as a visualization of correct and incorrect counts.



## Per-class evaluation

Per-class precision, recall, F1, and support are used to identify categories that are consistently detected and categories for which the model struggles.

The per-class analysis is more informative than accuracy alone because a model can achieve reasonable global accuracy while still failing on smaller customer-interest classes.

The target imbalance observed during data construction therefore directly affects interpretation of the evaluation metrics.

## Business Interpretation

**What customer interest has been discovered**

The system discovers customer interests primarily from repeated historical category activity, purchasing frequency, recency, monetary behavior, payment behavior, review behavior, and review language.

The most informative conceptual pattern is not a single variable but the combination:

behavioral history + textual feedback

A customer repeatedly purchasing products from one category provides direct evidence of historical interest.

A customer with recent purchases, multiple transactions, and consistent category activity provides stronger behavioral evidence than a customer with very limited history.

Review text adds an additional layer of information by describing the customer's qualitative experience in natural language.

### **Personalized recommendation**

The prediction can be used as the first stage of a recommendation pipeline.

The system performs:

Customer → Interest Category → Relevant Products

The recommendation catalog is constructed from historical sales and review statistics.

Each product receives:

*RecommendationScore*

$$= 0.55 \times SalesScore + 0.30 \times RatingScore + 0.15 \times OrderScore$$

The largest weight is assigned to sales because repeated demand is a strong evidence of product relevance. Review quality receives the second-largest weight, while the number of unique orders provides an additional measure of product coverage.

The catalog contains 2,182 products.

### **Targeted promotion**

Predicted interest can be used to select promotional content.

For example, instead of presenting the same product category to every customer, the system can select products associated with the predicted interest category.

This creates a more customer-specific marketing signal.

### **Customer retention**

The behavioral representation contains recency and purchase-frequency information.

These features can help identify changes in activity.

A customer with decreasing activity and long purchase gaps may represent a different retention opportunity from an active repeat customer.

### **Customer value interpretation**

The secondary behavior model classifies target-order value into:

- Low
- Medium
- High

This output can be combined with predicted interest.

Conceptually:

$$\text{Customer profile} = \text{Interest} + \text{Value behavior}$$

A high-value customer interested in a particular category may justify a different recommendation or campaign strategy from a low-value customer with the same predicted category.

### **Review text contribution**

The current evidence shows an important but nuanced result.

Text is genuinely part of the final model representation. In the inference sensitivity test, changing only the review text changes the transformed representation by:

$$0.21108$$

in maximum absolute feature difference.

Maximum representation difference: 0.21108049342756888

|    | Class                       | Review A Probability | Review B Probability \ |
|----|-----------------------------|----------------------|------------------------|
| 0  | beleza_saude                | 0.085910             | 0.085928               |
| 1  | brinquedos                  | 0.032240             | 0.032268               |
| 2  | cama_mesa_banho             | 0.274980             | 0.274734               |
| 3  | esporte_lazer               | 0.060850             | 0.060179               |
| 4  | fashion_bolsas_e_acessorios | 0.012077             | 0.012087               |
| 5  | informatica_acessorios      | 0.045579             | 0.046101               |
| 6  | moveis_decoracao            | 0.181883             | 0.181084               |
| 7  | other                       | 0.183463             | 0.184656               |
| 8  | relogios_presentes          | 0.037908             | 0.037664               |
| 9  | telefonica                  | 0.006353             | 0.006461               |
| 10 | utilidades_domesticas       | 0.078758             | 0.078837               |

The resulting class probabilities also change, although only slightly in the tested example. The maximum probability change is:

0.00119

The predicted class remains the same in that particular test.

Prediction A: moveis\_decoracao  
Prediction B: moveis\_decoracao

Maximum probability difference: 0.001192705545763867

This demonstrates that review text is not merely a decorative input. The model uses it, but its influence is weaker than the dominant structured behavioral information for the tested customer.

At the validation level, the best combined model reaches Macro F1 of 0.4544, while the best Tabular Only model reaches 0.4705.

Therefore, the current experiment does not provide evidence that review text improves the best overall predictive performance.

This is still an informative finding because it shows that the predictive value of a data source should be measured empirically rather than assumed.

## Deployment finding

The final inference example produces:

- Predicted customer behavior: High
- Behavior confidence: 0.436
- Predicted customer interest: moveis\_decoracao
- Interest confidence: 0.2742

```
=====
FINAL INFERENCE RESULT
=====
```

```
Customer ID: 15a38a0c1bf789e06ec42d24e485e794
```

```
Review Text:
```

```
o forro interno é frágil e todo manchado de tinta o suporte de controle tem
grandes falhas na pintura interna
```

```
Predicted Customer Behavior: High
```

```
Behavior Confidence: 0.436
```

```
Predicted Customer Interest: moveis_decoracao
```

```
Interest Confidence: 0.2742
```

```
Top 5 Recommended Products:
```

```
Empty DataFrame
```

```
Columns: [product_id, category_en, sold_units, average_price,
average_review_score, recommendation_score]
```

```
Index: []
```

The recommendation engine returns an empty result for this particular demonstration customer.

The reason is an important implementation issue: the prediction uses the original Portuguese category label while the recommendation catalog is normalized to an English category field. The prediction category and recommendation category therefore do not always match exactly.

## 9 Deployment

### Deployment architecture

The intended system architecture is:

User Input → Validation → Saved Preprocessing → Saved SVM Model  
→ Predicted Interest → Behavior Prediction → Recommendation

The final interest model is persisted as:

interest\_model\_pipeline.joblib

The persisted pipeline contains both preprocessing and the trained SVM classifier.

Additional artifacts include:

- interest\_preprocessor.joblib
- behavior\_model\_pipeline.joblib
- behavior\_preprocessor.joblib
- recommendation\_catalog.joblib
- metadata.json

The model directory contains six persisted artifacts.

## **Final preprocessing pipeline**

The final interest pipeline performs:

Numerical Features → Median Imputation → Standard Scaling

Categorical Features → Most-Frequent Imputation → One-Hot Encoding

Review Text → Word TF-IDF + Character TF-IDF

These representations are combined before classification.

The final transformed sample has shape:

$$5 \times 6073$$

and is stored as a sparse CSR matrix.

## **Web application**

The intended web workflow is:

User Input → Validation → Preprocessing → Saved Model → Prediction → Result

For customer behavior, the interface should accept the customer variables represented in the trained model and return:

- predicted customer interest;
- interest confidence;
- predicted customer behavior;
- behavior confidence;
- recommended products.

The final output contract is represented in JSON form with customer identifier, behavior prediction, interest prediction, confidence values, and recommendations.

The screenshot shows a web interface titled "Customer Input". Below the title is a subtitle: "Required inputs are shown first. Other model features are optional." There are two buttons: "Load Sample Customer" and "Clear". Below these is a section titled "Required Inputs" with a red "REQUIRED" label. This section contains several input fields arranged in a grid. The first row has four fields: "Prior Order Count" (value: 5), "Recency Days" (value: 18), "Total Spending" (value: 620.5), and "Average Order Value" (value: 124.1). The second row has four fields: "Total Items" (value: 9), "Average Review Score" (value: 4.5), "Customer State" (value: SP), and "Dominant Payment Type" (value: credit\_card). Below the grid is a text area labeled "Review Text" with the value "Bad".

| Customer Input      |                        |                  |                         |
|---------------------|------------------------|------------------|-------------------------|
| Required Inputs     |                        |                  |                         |
| Prior Order Count * | Recency Days *         | Total Spending * | Average Order Value *   |
| 5                   | 18                     | 620.5            | 124.1                   |
| Total Items *       | Average Review Score * | Customer State * | Dominant Payment Type * |
| 9                   | 4.5                    | SP               | credit_card             |
| Review Text *       |                        |                  |                         |
| Bad                 |                        |                  |                         |

Other Model Inputs

OPTIONAL

Optional engineered features. Leave blank when unavailable.

|                                            |                                   |                                       |                                      |
|--------------------------------------------|-----------------------------------|---------------------------------------|--------------------------------------|
| Max Order Value                            | History Duration Days             | Average Days Between Orders           | Min Days Between Orders              |
| 245                                        | 142                               | 35.5                                  | 18                                   |
| Max Days Between Orders                    | Average Item Price                | Average Freight Value                 | Average Items Per Order              |
| 55                                         | 68.5                              | 15.2                                  | 1.8                                  |
| Category Count Beleza Saude                | Category Count Brinquedos         | Category Count Cama Mesa Banho        | Category Count Esporte Lazer         |
| 2                                          | 2                                 | 2                                     | 2                                    |
| Category Count Fashion Bolsas E Acessorios | Category Count Ferramentas Jardim | Category Count Informatica Acessorios | Category Count Moveis Decoracao      |
| 2                                          | 2                                 | 2                                     | 2                                    |
| Category Count Perfumaria                  | Category Count Relogios Presentes | Category Count Telefonica             | Category Count Utilidades Domesticas |
| 2                                          | 2                                 | 2                                     | 2                                    |
| Category Share Beleza Saude                | Category Share Brinquedos         | Category Share Cama Mesa Banho        | Category Share Esporte Lazer         |
| 0.2                                        | 0.2                               | 0.2                                   | 0.2                                  |
| Category Share Fashion Bolsas E Acessorios | Category Share Ferramentas Jardim | Category Share Informatica Acessorios | Category Share Moveis Decoracao      |
|                                            |                                   |                                       |                                      |

Category Share Fashion Bolsas E Acessorios

0.2

Category Share Ferramentas Jardim

0.2

Category Share Informatica Acessorios

0.2

Category Share Moveis Decoracao

0.2

Category Share Perfumaria

0.2

Category Share Relogios Presentes

0.2

Category Share Telefonica

0.2

Category Share Utilidades Domesticas

0.2

Average Installments

2.5

Average Payment Value

124.1

Review Count

4

Text Review Count

3

Predict Customer

Customer Behavior

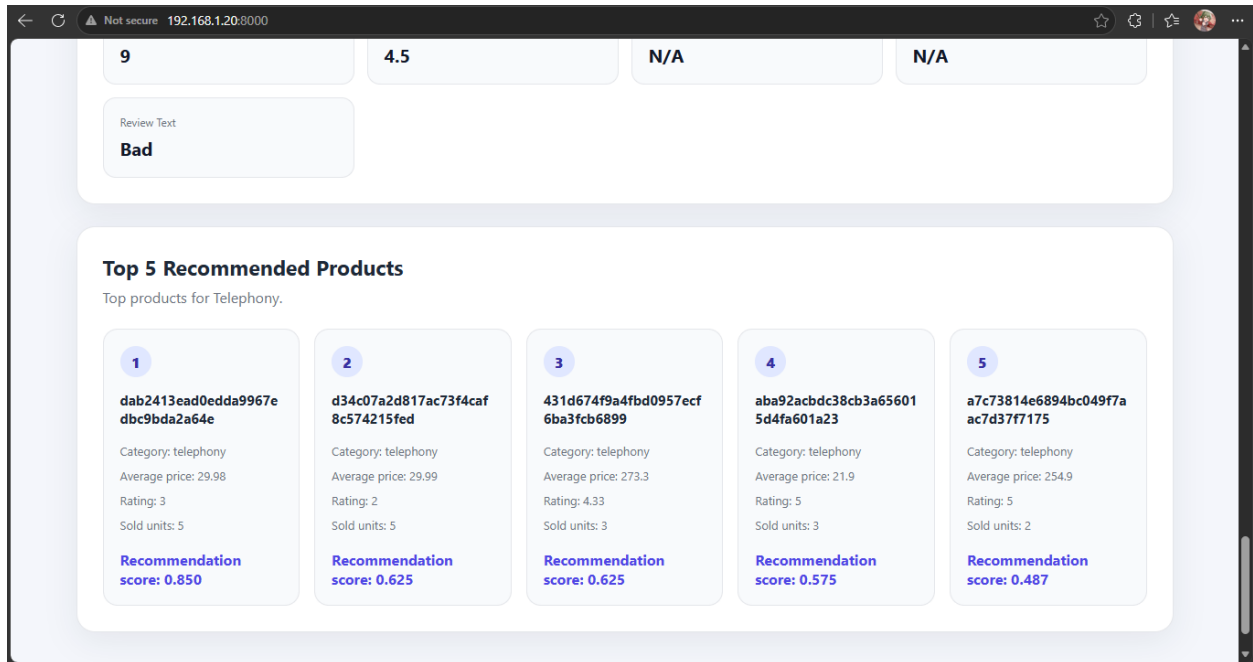
High

Confidence: 36.0%

Customer Interest

Telephony

Confidence: 46.6%



### 10.9.5 Mobile application

The mobile client follows the same inference service rather than retraining the model locally.

The expected architecture is:

Mobile UI → REST API → Saved Preprocessing → Saved Model → Prediction  
→ REST Response → Mobile UI

The mobile application should allow the user to enter or select customer information, submit the request, and display the predicted interest with confidence.

OLIST

Customer Intelligence

Predict customer behavior and interests.

Customer Information

Required Inputs

Prior Order Count

5.0

Recency Days

18.0

Total Spending

620.5

Average Order Value

124.1

Total Items

9.0

Average Review Score

4.5

Customer State

SP

Dominant Payment Type

credit\_card

Review Text

Good product, excellent quality, fast delivery and very satisfied with the purchase.

Load Sample

Clear

Predict Customer

Prediction Result

Customer Behavior

Medium

Confidence: 89.5%

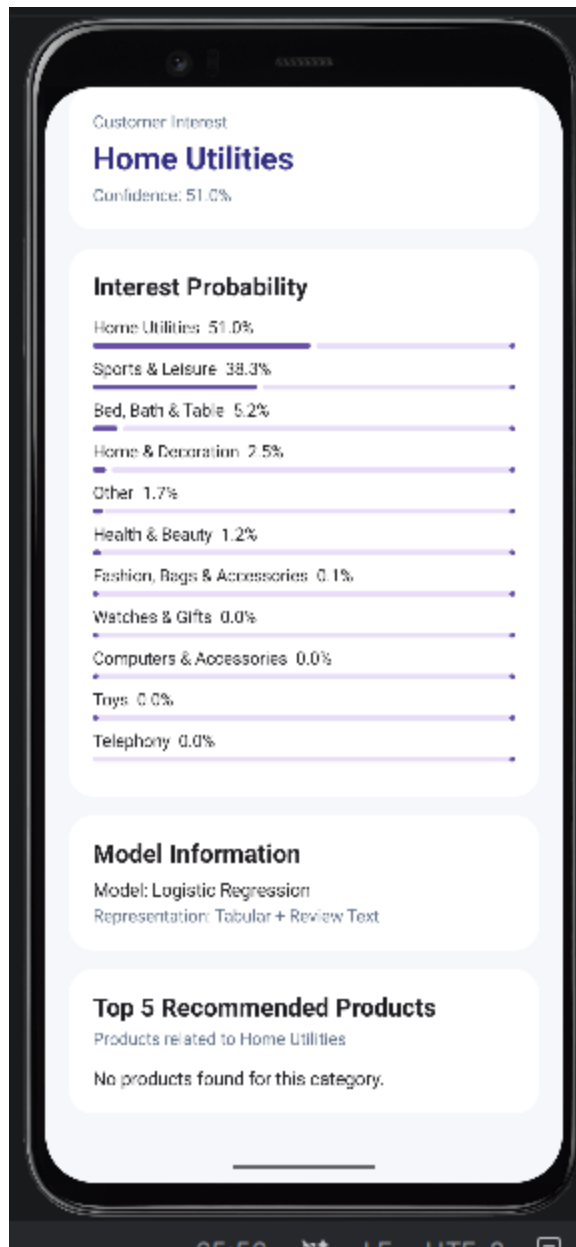
Customer Interest

Home Utilities

Confidence: 51.0%

Interest Probability

Home Utilities: 51.0%



### 10.9.6 Deployment reliability

The model persistence strategy is designed to make the system reproducible.

The saved model pipeline remembers the learned preprocessing configuration and model parameters.

This is essential because using a different preprocessing procedure during deployment could change the meaning of the numerical features and invalidate the model's learned relationships.

One particularly important example is TF-IDF.

The deployed service must use the same vocabulary and feature transformation learned during training.

The service must not build a new vocabulary from one customer's review text.